

PREFAȚĂ

Java este o tehnologie lansată de compania Sun Microsystems în 1995, impunându-se prin calități deosebite cum ar fi simplitate, robustețe și portabilitate.

Prezenta carte este o culegere de probleme rezolvate în limbajul Java. Lucrarea este adresată studenților de la facultăți tehnice, în scopul învățării cât mai temeinice a programării orientate pe obiecte.

Cartea cuprinde un număr de 100 de programe rezolvate de către autori și este structurată pe șapte capitole:

- Instrucțiuni elementare (atribuire, decizie, repetare)
- Dezvoltarea și utilizarea de clase
- Vectori. Matrici
- Șiruri de caractere
- Clase derivate. Clase abstracte. Polimorfism
- Fișiere
- Aplicații grafice

Pitești, 2007

Prof. dr. ing. Alexandru ENE
Universitatea din Pitești

CUPRINS

Prefață	
Instrucțiuni elementare (atribuire, decizie, repetare)	7
Dezvoltarea și utilizarea de clase	16
Vectori. Matrici	31
Șiruri de caractere	58
Clase derivate. Clase abstracte. Polimorfism	66
Fișiere	82
Aplicații grafice	109
Bibliografie	


```
    }  
  }  
  if(are==false)System.out.println("nu are solutie");  
}  
}
```

3. Să se calculeze valoare constantei PI, pe baza formulei:

$$\pi/4 = 1 - 1/3 + 1/5 - 1/7 + 1/9 - \dots$$

Se vor lua N=3000 de termeni în această sumă.

```
class Pi  
{  
  public static void main(String args[])  
  {  
    final int N=3000;//nr. de termeni ce se aduna  
    double pi=0;  
    for(int i=0;i<N;i++)  
      if(i%2==0)pi=pi+4.0/(2*i+1);  
      else pi=pi-4.0/(2*i+1);  
    System.out.println("PI calculat ca suma de "+N+" termeni: "+pi);  
    System.out.println("Constanta PI din clasa Math: "+Math.PI);  
  }  
}
```

4. Să se afișeze toți divizorii unui număr întreg dat ca parametru în linia de comandă.

```
class Divizori  
{  
  public static void main(String args[])  
  {  
    int nr=Integer.parseInt(args[0]);  
    for(int i=1; i<=nr; i++)  
      if(nr%i ==0)System.out.println(i);  
  }  
}
```

5. Să se calculeze primul număr prim mai mare decât un număr dat ca parametru în linia de comandă.

```
class PrimulNrPrim  
{  
  public static void main(String args[])
```

```
{
    int nr=Integer.parseInt(args[0]);
    int nrCrt=nr+1;//cautam incepand cu nr+1
    for(;;){
        if(estePrim(nrCrt))
            break;//l-a gasit
        else nrCrt++;
    }
    System.out.println("Primul numar prim mai mare este: "+nrCrt);
}
private static boolean estePrim(int x)
{
    boolean este=true;
    for(int i=2;i<=Math.sqrt(x);i++){
        if(x % i == 0){
            este=false;
            break; }
    }
    return este;
}
```

6. Se citesc de la tastatură trei numere întregi a, b și c. Să se ordoneze aceste numere crescătoare, astfel încât a va fi minimul, iar în c va fi maximul. Citirea de la tastatură se va face cu metoda `showInputDialog()` din clasa `JOptionPane`.

```
import javax.swing.*;
class Ordonare3
{
    public static void main(String args[])
    {
        int a=Integer.parseInt(JOptionPane.showInputDialog("a="));
        int b=Integer.parseInt(JOptionPane.showInputDialog("b="));
        int c=Integer.parseInt(JOptionPane.showInputDialog("c="));
        if(a>b){
            // le comutam:
            int aux=a; a=b; b=aux;
        }
        if(b>c){
            //le comutam:
            int aux=b; b=c; c=aux;
        }
    }
}
```

```

    }
    if(a>c){
        //le comutam:
        int aux=a;a=c; c=aux;
    }

    System.out.println(a+" "+b+" "+c);
}
}

```

7. Se citesc două numere a și b. Să se afișeze dacă cele două numere au același număr total de divizori.

```

import javax.swing.*;
class Comparare
{
    public static void main(String args[])
    {
        int a=Integer.parseInt(JOptionPane.showInputDialog("a="));
        int b=Integer.parseInt(JOptionPane.showInputDialog("b="));
        int na=2; // numarul de divizori ai lui a
        // orice numar are ca divizori pe 1 si el insusi
        for(int i=2;i<=a/2;i++)
            if(a%i==0)na++;
        int nb=2; // numarul de divizori ai lui b
        for(int i=2;i<=b/2;i++)
            if(b%i==0)nb++;
        if(na>nb)System.out.println(a+" are mai multi divizori ");
        else if(na==nb)System.out.println("acelasi numar de divizori ");
        else System.out.println(b+" are mai multi divizori ");
    }
}

```

8. Se citește un număr natural a. Să se afișeze dacă este pătrat perfect sau nu.

```

import javax.swing.*;
class PatratPerfect
{
    public static void main(String args[])
    {
        int a=Integer.parseInt(JOptionPane.showInputDialog("a="));
    }
}

```

```
double radical=Math.sqrt(a);
if((int)radical*(int)radical==a)
    System.out.println("este");
else System.out.println("nu este");
}
}
```

9. Se citesc n_1 și n_2 capetele unui interval închis. Să se afișeze dacă în intervalul $[n_1, n_2]$, există cel puțin un număr prim.

```
import javax.swing.*;
class ExistaNumerePrime
{
    public static void main(String args[])
    {
        int n1=Integer.parseInt(JOptionPane.showInputDialog("n1="));
        int n2=Integer.parseInt(JOptionPane.showInputDialog("n2="));
        boolean exista=false;
        for(int i=n1;i<=n2;i++)
            if(estPrim(i)){
                exista=true;
                break;
            }
        if(exista)System.out.println("exista");
        else System.out.println("nu exista");
    }
    private static boolean estPrim(int nr){
        for(int i=2;i<=Math.sqrt(nr);i++)
            if(nr%i==0)return false;
        return true;
    }
}
```

10. Se citește un număr natural de la tastatură. Să se afișeze care este cifra maximă din acest număr.

```
import javax.swing.*;
class CifraMaxima
{
    public static void main(String args[])
    {
        int a=Integer.parseInt(JOptionPane.showInputDialog("a="));
```

```
int max=0;
for(;;){
    int cifraCrt=a%10;
    if(cifraCrt>max)max=cifraCrt;
    a=a/10;//reduc numarul:
    if(a==0)break;
}
System.out.println(max);
}
}
```

11. Se citesc trei numere întregi *a*, *b* și *c*, de la tastatură. Să se calculeze maximul dintre *a* și *b* (se va folosi metoda definită separat *maxim()*, ce are doi parametrii) și maximul dintre *a*, *b* și *c* (se va folosi metoda definită separat *maxim()*, ce are trei parametrii). Această aplicație ilustrează polimorfismul parametric în Java (posibilitatea ca în aceeași clasă să existe două sau mai multe metode cu același nume, dar cu liste de parametrii diferiți).

```
import javax.swing.*;
class CalculMaxim
{
    public static void main(String args[])
    {
        int a=Integer.parseInt(JOptionPane.showInputDialog("a="));
        int b=Integer.parseInt(JOptionPane.showInputDialog("b="));
        int c=Integer.parseInt(JOptionPane.showInputDialog("c="));
        int max2=maxim(a,b);
        System.out.println(max2);
        int max3=maxim(a,b,c);
        System.out.println(max3);
    }
    private static int maxim(int a,int b){
        if(a>=b)return a;
        else return b;
    }
    private static int maxim(int a,int b,int c){
        int max=a;
        if(b>max)max=b;
        if(c>max)max=c;
        return max;
    }
}
```

```
}  
}
```

12. Se citește un număr natural n . Se citesc primul termen și rația unei progresii aritmetice. Să se calculeze, recursiv, termenul de rang n al progresiei.

```
import javax.swing.*;  
class ProgresieAritmetica  
{  
    public static void main(String args[])  
    {  
        double a1=Double.parseDouble(JOptionPane.showInputDialog(  
                                                    "primul termen al progresiei="));  
        double r=Double.parseDouble(JOptionPane.showInputDialog(  
                                                    "ratia ="));  
        int n=Integer.parseInt(JOptionPane.showInputDialog("n ="));  
        //calcul termen de rang n:  
        double a_n=calcul(a1,r,n);  
        System.out.print("Termenul de rang "+n+" este: "+a_n);  
    }  
    private static double calcul(double primulTermen, double ratia,  
                                int n)  
    {  
        if(n==1)return primulTermen;  
        return ratia+calcul(primulTermen, ratia, n-1);  
    }  
}
```

13. Se citește un număr natural N . Să se calculeze termenul de rang N din șirul lui Fibonacci.

Șirul lui Fibonacci se definește recursiv astfel:

$a_0 = 1$

$a_1 = 1$

$a_n = a_{n-1} + a_{n-2}$, pentru $n \geq 2$

```
import javax.swing.*;
```

```
class Fibonacci
```

```
{  
    public static void main(String args[])  
    {  
        int N=Integer.parseInt(JOptionPane.showInputDialog("N="));
```

```

    System.out.println(fib(N));
}
private static int fib(int n){
    if(n==0)return 1;
    if(n==1)return 1;
    return fib(n-1)+fib(n-2);
}
}

```

14. Se citește un număr natural N. Să se afișeze dacă acest număr este termen în șirul lui Fibonacci.

```

import javax.swing.*;
class EsteFibonacci
{
    public static void main(String args[])
    {
        int x=Integer.parseInt(JOptionPane.showInputDialog("x="));
        boolean este=false;
        int n=0;
        for(;;){
            int termen=fib(n);
            if(x==termen){
                este=true;
                break;
            }
            if(x<termen)break; //deci, nu este
            // trecem la urmatorul termen Fibonacci:
            n++;
        }
        if(este)System.out.println("este");
        else System.out.println("nu este");
    }
    private static int fib(int n){
        if(n==0)return 1;
        if(n==1)return 1;
        return fib(n-1)+fib(n-2);
    }
}

```

15. Se citește un număr natural N. Să se calculeze suma primilor N termeni din șirul lui Fibonacci.

```
import javax.swing.*;
class SumaFib
{
    public static void main(String args[])
    {
        int N=Integer.parseInt(JOptionPane.showInputDialog("N="));
        int suma=0;
        for(int i=0;i<N;i++)
            suma=suma+fib(i);
        System.out.println(suma);
    }
    private static int fib(int n){
        if(n==0)return 1;
        if(n==1)return 1;
        return fib(n-1)+fib(n-2);
    }
}
```

Cap. 2

Dezvoltarea și utilizarea de clase elementare

1. Să se afișeze din câte încercări se generează trei numere aleatoare egale, în gama 0..19.

```
import java.util.*;
class NrIncerari{
public static void main(String args[]){
    final int GAMA=20;
    Random r=new Random();
    int contor=0;
    for(;;){
        int a=r.nextInt(GAMA);
        int b=r.nextInt(GAMA);
        int c=r.nextInt(GAMA);
        contor++;
        if((a==b)&&(b==c))break;
    }
    System.out.println(contor);
}
}
```

2. Folosind clasa BigInteger să se calculeze 2^{1000} . Vom folosi metoda pow() din această clasă. Să se afișeze și câte cifre are acest rezultat. Metoda pow() are semnătura:

```
public BigInteger pow(int exponent)
import java.math.*;
class BigInteg1
{
public static void main(String args[])
{
    BigInteger baza=new BigInteger("2");
    BigInteger rezultat=baza.pow(1000);
    String s=rezultat.toString();
    System.out.println("rezultat="+s);
    System.out.println("nr. cifre="+s.length());
}
}
```

3. Folosind clasa `BigInteger`, să se calculeze și afișeze valoarea numărului 50!. Se va folosi metoda `multiply()`, ce are ca parametru un număr `BigInteger` și scoate ca rezultat un număr de tip `BigInteger`, care reprezintă rezultatul înmulțirii între numărul dat ca parametru și obiectul curent (pentru care se apelează metoda `multiply()`).

```
import java.math.*;
class Fact50
{
    public static void main(String args[])
    {
        BigInteger rez=new BigInteger("1");
        for(int i=2;i<=50;i++)
            rez=rez.multiply(new BigInteger(""+i));
        System.out.println(rez);
    }
}
```

4. Se dă următorul fragment de program:

```
class A
{
    public int x;
    ...
}
class B
{
    public static void main(String args[])
    {
        ...
    }
}
```

În metoda `main()`, să se scrie în variabila `x` din clasa `A`, valoarea 7.

```
public static void main(String args[])
{
    A obiect=new A();
    obiect.x=7;
    .....
}
```

5. Se dă următorul fragment de program:

```
class A
```

```

{
    public int x;
    private double y;
    ...
}
class B
{
    public static void main(String args[])
    {
        //scriem in variabilele x si y din clasa A:
        x=3;
        y=7;
        ...
    }
}

```

Să se explice de ce cele două atribuiri din metoda main() sunt greșite.

Explicații:

În variabila publică x, nu se poate scrie direct din afara clasei ei. Trebuie mai întâi instanțiat un obiect din clasa A, și pe baza lui se accesează variabila x, astfel:

A obj=new A(); obj.x=3;

În cazul variabilei private y, fiind private, nu avem acces din afara clasei în care ea este definită.

6. Să se scrie o metodă ce are ca parametru un număr natural N; metoda returnează două numere aleatoare diferite, în gama 0...N-1. Dați și un exemplu de aplicație în care se folosește această metodă.

```
import java.util.*;
```

```
class Generare2Aleatoare
```

```

{
    public static void main(String args[])
    {
        final int N=100;
        Dublet d=getDouaNumereDiferite(N);
        //afisam numerele:
        System.out.println(d.x);
        System.out.println(d.y);
    }
    //metoda ce returneaza doua numere aleatoare
    //diferite in gama 0..N-1
}

```

```
private static Dublet getDouaNumereDiferite(int N)
{
    Random r=new Random();
    int n1=r.nextInt(N);//primul numar
    int n2;
    for(;;){
        n2=r.nextInt(N);
        if(n1!=n2)break;
    }
    //formez un obiect din clasa Dublet:
    Dublet d2=new Dublet(n1,n2);
    return d2;
}
}
class Dublet
{
    public int x;
    public int y;
    public Dublet(int a, int b)
    {
        x=a;
        y=b;
    }
}
```

7. Pentru trei numere întregi a, b și c, să se calculeze maximul, minimul și media aritmetică a celor trei numere, folosind o metodă separată ce are ca parametrii trei numere întregi și care returnează trei rezultate: maximul, minimul și media aritmetică a celor trei numere.

```
class Calcul3Rez
{
    public static void main(String args[])
    {
        //Initializam in mod direct cele trei numere:
        int a=7;
        int b=8;
        int c=4;
        Triplet t=getRezultate(a,b,c);
        //afisam numerele:
        System.out.println("minim="+t.x);
    }
}
```

```
System.out.println("maxim="+t.y);
System.out.println("medie="+t.z);
}
//metoda ce returneaza cele trei rezultate:
private static Triplet getRezultate(int a, int b, int c)
{
    int min=a; if(b<min)min=b; if(c<min)min=c;
    int max=a; if(b>max)max=b; if(c>max)max=c;
    double medie=(a+b+c)/3.0;
    //formez un obiect din clasa Triplet:
    Triplet t=new Triplet(min, max, medie);
    return t;
}
}
class Triplet
{
    public int x;
    public int y;
    public double z;
    public Triplet(int a, int b, double c)
    {
        x=a;
        y=b;
        z=c;
    }
}
```

8. Scrieți clasa Numar ce are ca variabilă de instanță privată un număr întreg nr, și ca metode:

- constructorul, ce inițializează variabila nr;
- getNr(), ce returnează valoarea variabilei private nr;
- metoda estePatratPerfect() ce returnează *true* dacă nr este pătrat perfect și *false* în caz contrar;
- metoda estePrim() ce returnează *true* dacă nr este prim și *false* în caz contrar;
- metoda afisareDivizori() ce afișează divizorii numărului nr.

Scrieți și o clasă de test pentru clasa Numar.

```
class Numar
{
    private int nr;
```

```
public Numar(int x)
{
    nr=x;
}
public int getNr()
{
    return nr;
}
public void afisareDivizori()
{
    //nr. 1, este sigur divizor:
    System.out.println("1");
    //Cautam divizorii intre 2 si jumatatea numarului:
    for(int i=2;i<=nr/2;i++)
        if(nr%i==0)System.out.println(i);
    //numarul se divide cu el insusi:
    System.out.println(nr);
}
public boolean estePrim()
{
    boolean este = true;
    for(int i=2;i<=Math.sqrt(nr);i++)
        if(nr%i==0){
            este = false;
            break;
        }
    return este;
}
public boolean estePatratPerfect()
{
    int radical=(int)Math.sqrt(nr);
    if(radical*radical==nr)return true;
    else return false;
}
}

class TestNumar
{
    public static void main (String args[])
    {
```

```

    Numar n=new Numar(25);
    System.out.println("Divizorii numarului "+n.getNr());
    n.afisareDivizori();
    System.out.println("este patrat perfect="+n.estePatratPerfect());
    if(n.estePrim()==true)System.out.println(n.getNr()+" este prim.");
    else System.out.println(n.getNr()+" nu este prim.");
}
}

```

9. Să se construiască clasa Dreptunghi, ce are ca variabile de instanță private, două numere întregi a și b, ce reprezintă lungimile laturilor unui dreptunghi. În această clasă avem ca metode:

- constructorul, ce face inițializările;
- metoda calculPerimetru(), ce returnează perimetrul dreptunghiului;
- metoda calculArie(), ce returnează aria dreptunghiului;
- metoda estePatrat(), ce returnează *true* dacă dreptunghiul este pătrat;
- metoda suntEgale(), ce are ca parametru un dreptunghi d și scoate ca rezultat *true* dacă dreptunghiul curent (cel pentru care se apelează metoda) este egal cu dreptunghiul d.

Scrieți și o clasă de test pentru clasa Dreptunghi.

```

class Dreptunghi
{
    private int a;
    private int b;
    public Dreptunghi(int x,int y)
    {
        a=x;
        b=y;
    }
    public int calculPerimetru()
    {
        return 2*(a+b);
    }
    public int calculArie()
    {
        return a*b;
    }
}

```

```
public boolean estePatrat()
{
    if(a==b)return true;
    else return false;
}

public boolean suntEgale(Dreptunghi d)
{
    if ((this.a==d.a)&&(this.b==d.b))return true;
    else return false;
}
}
class TestDreptunghi
{
    public static void main (String args[])
    {
        Dreptunghi d=new Dreptunghi(5,7);
        System.out.println("Perimetrul este "+d.calculPerimetru());
        System.out.println("Aria este "+d.calculArie());
        System.out.println("Dreptunghiul este patrat= "+d.estePatrat());
        Dreptunghi d1=new Dreptunghi(5,7);
        System.out.println("Sunt egale= "+d.suntEgale(d1));
    }
}
```

10. Să se dezvolte clasa DouaNumere, ce are variabile de instanță două numere întregi a și b, și ca metode:

- constructorul ce face inițializările;
- metoda maxim(), ce returnează maximul dintre a și b;
- metoda cmmdc(), ce returnează cel mai mare divizor comun al celor două numere.

Scrieți și o clasă de test pentru clasa DouaNumere.

```
class DouaNumere
{
    private int a,b,c;
    public DouaNumere(int n1, int n2)
    {
        a=n1;
        b=n2;
    }
}
```

```
public int maxim()
{
    if(a>b)return a;
    else return b;
}
public int cmmdc()
{
    while(a!=b)
        if(a>b)a=a-b;
        else b=b-a;
    return a;
}
}
class Test2Numere
{
    public static void main(String args[])
    {
        DouaNumere d=new DouaNumere(12,18);
        System.out.println("maximul este: "+d.maxim());
        System.out.println("cmmdc este: "+d.cmmdc());
    }
}
```

11. Să se dezvolte clasa *TreiNumere*, ce are variabile de instanță trei numere întregi a,b și c, și ca metode:

- constructorul ce face inițializările;
- metoda *maxim()*, ce returnează maximul dintre a, b și c;
- metoda *suntPitagorice()*, ce returnează *true*, dacă a, b, c sunt numere pitagorice.

Scrieți și o clasă de test pentru clasa *TreiNumere*.

```
class TreiNumere
{
    private int a,b,c;
    //constructor:
    public TreiNumere(int n1, int n2, int n3)
    {
        a=n1;
        b=n2;
        c=n3;
    }
}
```

```
public int maxim()
{
    int max=a;
    if(b>max)max=b;
    if(c>max)max=c;
    return max;
}
public boolean suntPitagorice()
{
    if((a*a==b*b+c*c)|| (b*b==a*a+c*c)|| (c*c==a*a+b*b))
        return(true);
    else return(false);
}
}
class Test3Numere
{
    public static void main(String args[])
    {
        TreiNumere t=new TreiNumere(3,5,4);
        System.out.println("maximul este: "+t.maxim());
        if(t.suntPitagorice())System.out.println("Sunt numere pitagorice");
        else System.out.println("Nu sunt numere pitagorice");
    }
}
```

12. Să se dezvolte clasa Complex, ce are variabile de instanță private două numere întregi re și im (partea reală și partea imaginară a unui număr complex) și ca metode:

- constructorul ce face inițializările;
 - modul(), ce returnează modulul numărului complex;
 - suma(), ce are ca parametru un număr complex c, prin care la numărul complex curent se adună numărul complex c (rezultatul se depune în numărul curent);
 - produs(), ce are ca parametru un număr complex c, prin care în numărul complex curent se depune rezultatul înmulțirii dintre numărul complex curent și numărul complex c;
 - getRe(), ce returnează partea reală a numărului complex;
 - getIm(), ce returnează partea imaginară a numărului complex;
-

- equals(), ce redefinește metoda equals() din clasa Object, prin care se compară din punct de vedere al conținutului, două obiecte Complex: obiectul curent și obiectul dat ca parametru;
- toString(), ce redefinește metoda toString() din clasa Object, prin care se dă o reprezentare sub formă de String a unui număr complex;

Scrieți și o clasa de test pentru clasa Complex.

```
class Complex
{
    private double re;
    private double im;
    public Complex(double x, double y)
    {
        re=x;
        im=y;
    }
    public double getRe()
    {
        return re;
    }
    public double getIm()
    {
        return im;
    }
    public double modul()
    {
        return Math.sqrt(re*re+im*im);
    }
    //adunarea nr. complex curent, cu un alt nr. complex, cu depunerea
    //rezultatului in numarul complex curent:
    public void suma(Complex c)
    {
        re=re+c.re;
        im=im+c.im;
    }
    //inmultirea nr. complex curent, cu un alt nr. complex, cu depunerea
    //rezultatului in numarul complex curent:
    public void produs(Complex c)
    {
        re=re*c.re-im*c.im;
```

```
    im=re*c.im+im*c.re;
}

//redefinirea metodei equals() din clasa parinte Object:
//(trebuie sa se pastreze aceeasi semnatura:)
public boolean equals(Object obj)
{
    Complex c=(Complex)obj;
    if(c!=null)
        if((re==c.re)&&(im==c.im))return true;
    return false;
}

//redefinirea metodei toString() din clasa parinte Object:
public String toString()
{
    String s="("+re+", "+im+")";
    return s;
}
}

class TestComplex
{
    public static void main(String args[])
    {
        Complex c1=new Complex(1,1);
        System.out.println("Modulul este= "+c1.modul());
        Complex c2=new Complex(1,1);
        c1.suma(c2);
        System.out.println("suma = "+c1.toString());
        Complex c3=new Complex(1,1);
        System.out.println("sunt egale: "+c2.equals(c3));
    }
}
```

13. Scrieți clasa Unghi, ce are ca variabilă de instanță privată un număr întreg x, măsura în grade a unui unghi, și ca metode:

- constructorul;
 - suntComplementare(), ce are ca parametru un alt unghi u, și care returnează *true* dacă unghiul u este complementar cu unghiul curent;
-

- `conversieRadiani()`, ce returnează valoarea exprimată în radiani a unghiului curent `x`.

Scrieți și o clasă de test pentru clasa `Unghi`.

```
class Unghi
{
    private int x;
    public Unghi(int x)
    {
        this.x=x;
    }
    public boolean suntComplementare(Unghi u)
    {
        if(this.x+u.x==90) return true;
        else return false;
    }
    public double conversieRadiani()
    {
        return (Math.PI*x)/180;
    }
}

class TestUnghi
{
    public static void main (String args[])
    {
        Unghi a=new Unghi(30);
        System.out.println("Radiani= "+a.conversieRadiani());
        System.out.println("sunt
        complementare="+a.suntComplementare(new Unghi(60)));
    }
}
```

14. Să se dezvolte clasa `Timp` ce are ca variabile de instanță trei numere întregi: `h`, `min`, `sec` (ore, minute, secunde). Ca metode:

- constructorul;
 - `conversieInSecunde()` ce returnează timpul curent, exprimat în secunde;
 - `compara()`, ce are ca parametru un `Timp t`, și care returnează 1 dacă timpul curent este mai mare ca `t`, 0 dacă cei doi timpi sunt egali , și -1 în caz contrar.
-

Scrieți și o clasă de test.

```
class Timp
{
    private int h; //ore
    private int min;
    private int sec;
    public Timp(int ore,int m, int s)
    {
        h=ore;
        min=m;
        sec=s;
    }
    public int conversieSecunde()
    {
        return 3600*h+60*min+sec;
    }
    public int compara(Timp t)
    //returneaza 0 daca cei doi timpi sunt egali
    //returneaza 1 daca timpul curent > timpul t dat ca argument
    //returneaza -1 daca timpul curent < timpul t
    {
        int sec1=this.conversieSecunde();//timpul curent, in secunde
        int sec=t.conversieSecunde();//timpul t, in secunde
        if(sec1>sec)return 1;
        else if(sec1==sec)return 0;
        else return -1;
    }
}

class TestTimp
{
    public static void main(String args[])
    {
        Timp t1=new Timp(1,5,6);
        Timp t2=new Timp(1,5,6);
        int rezultat=t1.compara(t2);
        if(rezultat==1)System.out.println("t1>t2");
        else if(rezultat==0)System.out.println("t1=t2");
        else System.out.println("t1<t2");
    }
}
```

```
}
```

15. Să se construiască clasa Cerc, ce are ca variabilă de instanță privată, un număr întreg r, ce reprezintă raza unui cerc. În această clasă avem ca metode:

- constructorul, ce face inițializarea razei;
- metoda calculPerimetru(), ce returnează perimetrul cercului;
- metoda calculArie(), ce returnează aria cercului;

Scrieți și o clasă de test pentru clasa Cerc.

```
class Cerc
{
    private int raza;
    public Cerc(int x)
    {
        raza=x;
    }
    public double calculPerimetru()
    {
        return 2*Math.PI*raza;
    }
    public double calculArie()
    {
        return Math.PI*raza*raza;
    }
}
class TestCerc
{
    public static void main (String args[])
    {
        Cerc c=new Cerc(3);
        System.out.println("Perimetru= "+c.calculPerimetru());
        System.out.println("Aria= "+c.calculArie());
    }
}
```

Cap. 3

Vectori. Matrici

1. Se citește de la tastatură un număr natural N ; se instanțiază un vector de N numere întregi. Să se completeze acest vector cu numere aleatoare în gama $0..N-1$, cu condiția ca fiecare număr din această gamă să apară o singură dată.

Algoritm: vom inițializa vectorul cu numerele $0, 1, \dots, N-1$, date în această ordine. Apoi, aceste numere inițiale, le vom comuta, pozițiile de comutare fiind generate aleator.

```
import javax.swing.*;
import java.util.*;
class InitNumAleatoare
{
    public static void main(String args[])
    {
        int N;
        String s = JOptionPane.showInputDialog("N=");
        N = Integer.parseInt(s);
        int a[] = new int [N];
        int i;
        //se initializeaza vectorul cu numerele 0,1,...,N-1, in aceasta ordine:
        for(i=0;i<a.length;i++)
            a[i] = i;
        Random r = new Random();
        //se repeta de N ori:
        for(i=0;i<N;i++){
            //se genereaza doua numere aleatoare n1 si n2:
            int n1 = r.nextInt(N);
            int n2 = r.nextInt(N);
            //se comuta variabilele a[n1] si a[n2]:
            int aux = a[n1];
            a[n1] = a[n2];
            a[n2] = aux;
        }
        //Afisare vector generat:
        for (i=0;i<N;i++)
            System.out.println(a[i]);
        System.exit(0);
    }
}
```

```
}
}
```

2. Se citește de la tastatură un număr natural N ; se instanțiază un vector de N numere întregi. Să se completeze acest vector cu numere aleatoare în gama $0..N-1$, cu condiția ca fiecare număr din această gamă să apară o singură dată. (Problema anterioară, dar un alt algoritm: vom introduce pe rând în vector, câte un număr aleator, cu condiția ca să fie diferit de numerele anterioare introduse).

```
import java.util.*;
class Aleatoare
{
    public static void main(String args[])
    {
        final int N=10;
        int a[]=new int[N];
        Random r=new Random();
        int i;
        int nr;
        for(i=0;i<N;i++){
            for(;;){
                nr=r.nextInt(N);//intre 0 si (N-1)
                //a mai fost generat?
                //cautare liniara:
                boolean estePrezent=false;
                for(int j=0;j<i;j++){
                    if(nr==a[j]){
                        estePrezent=true;
                        break;}
                if(estePrezent==false){
                    a[i]=nr;
                    break;
                }
            }
        }
        //afisare:
        for(i=0;i<N;i++)
            System.out.print(a[i]+" ");
        System.out.println("\n");
    }
}
```

```
}
```

3. Se citesc de la tastatură doi vectori *a* și *b*, ce au aceeași dimensiune (dimensiunea lor comună, *N*, este cunoscută). Să se afișeze dacă vectorul *b* este o permutare a vectorului *a*.

```
import javax.swing.*;
import java.util.*;
class SuntPermutari
{
    public static void main(String args[])
    {
        int N;
        String s = JOptionPane.showInputDialog("N=");
        N = Integer.parseInt(s);
        int a[] = new int [N];
        int b[] = new int [N];
        int i;
        for(i=0;i<N;i++)
            a[i]=Integer.parseInt(JOptionPane.showInputDialog("a["+i+"]="));
        for(i=0;i<N;i++)
            b[i]=Integer.parseInt(JOptionPane.showInputDialog("b["+i+"]="));
        //sortam cei doi vectori:
        Arrays.sort(a);
        Arrays.sort(b);
        //comparam cei doi vectori:
        boolean suntPerm=true;
        for(i=0;i<N;i++)
            if(a[i]!=b[i]){
                suntPerm=false;
                break;
            }
        if(suntPerm)System.out.println("sunt");
        else System.out.println("nu sunt");
    }
}
```

4. În clasa *System*, care face parte din pachetul *java.lang*, este definită metoda: *currentTimeMillis()* cu semnătura:
`public static long int currentTimeMillis()`.

Această metodă returnează valoarea timpului sistemului, în milisecunde.

Se generează $N = 100000$ numere întregi aleatoare, în gama $0 \dots 9999$, care se memorează în vectorul a . Folosind metoda `currentTimeMillis()`, să se calculeze durata sortării în ordine crescătoare a vectorului a . Se va sorta vectorul mai întâi cu metoda `Arrays.sort()`, și apoi același vector cu metoda de sortare prin interschimbare.

```
import javax.swing.*;
import java.util.*;
class DurataSortare
{
    public static void main(String args[])
    {
        final int N=100000;
        final int GAMA=1000;
        int a[] = new int [N];
        Random r=new Random();
        int i;
        for(i=0;i<N;i++)
            a[i]=r.nextInt(GAMA);
        //salvam vectorul a:
        int b[] = new int [N];
        for(i=0;i<N;i++)
            b[i]=a[i];
        long t1=System.currentTimeMillis();
        Arrays.sort(a);
        long t2=System.currentTimeMillis();
        System.out.println(t2-t1);
        //Acelasi vector sortat prin interschimbare:
        long t3=System.currentTimeMillis();
        sortare(b);
        long t4=System.currentTimeMillis();
        System.out.println("sortare interschimbare : "+(t4-t3));
    }
    private static void sortare(int a[])
    {
        for(int i=0;i<a.length-1;i++)
            for(int j=i+1;j<a.length;j++)
                if(a[i]>a[j]){
```

```
        int aux=a[i];
        a[i]=a[j];
        a[j]=aux;
    }
}
}
```

5. Să se scrie o aplicație ce folosește o metodă ce are doi parametri: un vector de numere întregi *a* , și un parametru de tip caracter denumit *sens*, ce poate avea două valori: 's' (stânga) și 'd' (dreapta). În metodă se rotesc în vectorul *a* toate elementele, cu o poziție, spre stânga sau spre dreapta, după valoarea parametrului *sens*.

Exemplu:

a={1,2,30} și *sens*='d'. După execuția metodei avem: *a*={30, 1, 2}

```
class Rotire
{
    public static void main(String args[])
    {
        int a[]={1,2,30};
        rotire(a,'d');
        for(int i=0;i<a.length;i++)
            System.out.print(a[i]+" ");
    }
    private static void rotire(int a[],char sens)
    {
        int N=a.length;//dimensiune vector a[]
        if(sens=='s'){
            int aux=a[0];
            for(int i=0;i<N-1;i++)
                a[i]=a[i+1];
            a[N-1]=aux;
        }
        else if(sens=='d'){
            int aux=a[N-1];
            for(int i=N-2;i>=0;i--)
                a[i+1]=a[i];
            a[0]=aux;
        }
    }
}
```

6. Scrieți o aplicație în care se implementează algoritmul de căutare binară a prezenței unui număr x citit de la tastatură într-un vector sortat crescător.

```
class CautareBinara
{
    public static void main(String args[])
    {
        int a[]={1,5,7,9,12,30};
        int x=1;
        //Este prezent x in vectorul a[] ?
        if(estePrezent(x,a,0,a.length-1))System.out.println("Este prezent.");
        else System.out.println("Nu este prezent.");
    }

    private static boolean estePrezent(int x,int a[],int s,int d)
    {
        if(s==d){
            if(x==a[s])return true;else return false;}
        if(d-s==1){
            if((x==a[s])||(x==a[d]))return true; else return false;}
        int m;
        m=(s+d)/2;
        if(x==a[m])return true;
        else if(x<a[m])return estePrezent(x,a,s,m-1);
        else return estePrezent(x,a,m+1,d);
    }
}
```

7. Scrieți o aplicație Java în care se implementează algoritmul de sortare prin metoda selecției maximului.

```
import javax.swing.*;
class SelectieMaxim{
    public static void main(String args[]){
        final int N=5;
        int A[]=new int[N];
        int i;
        //citire vector A[]:
        for(i=0;i<N;i++)
            A[i]=Integer.parseInt(JOptionPane.showInputDialog("nr="));
    }
}
```



```
ordonareCresc(A,N);
System.out.println("Numerele sortate crescator: ");
for(i=0;i<N;i++)
    System.out.println(A[i]);
} //main
private static void ordonareCresc(int A[], int nA)
//trebuie oblig. ca parametru dat si nA, pt. ca metoda se va apela
// recursiv pt. mai putine elemente.
{
    if(nA==1)return;//un singur element, este implicit ordonat!
    int poz=getPozMax(A,nA);//indexul maximului
    if(poz!=nA-1){//daca este chiar pe ultima poz., nu avem ce comuta!
        //comut elementul de pe ultima poz. din A cu maximul gasit:
        int temp=A[nA-1];
        A[nA-1]=A[poz];
        A[poz]=temp;}
    //ordoneaza recursiv portiunea din vectorul A[]
    // ce are doar nA-1 componente:
    ordonareCresc(A,nA-1);
} //ordonareCresc
private static int getPozMax(int A[], int nA)
{
    int max=A[0];
    int pozMax=0;
    for(int i=1;i<nA;i++)
        if(A[i]>max){
            max=A[i];
            pozMax=i;}
    return pozMax;
}
}
```

8. Scrieți o aplicație Java în care se implementează algoritmul de interclasare a doi vectori sortați crescător.

```
class Interclasare
{
    public static void main(String args[]){
        //Se dau doi vectori A[] si B[], fiecare ordonat crescator. Sa obtinem
        // vectorul C, ordonat crescator, ce cuprinde toate elementele din
        //primii doi vectori.
```

```
int A[]={1,10,200};
int B[]={5, 6, 7, 22, 33,4090};
int nA=A.length;//nr. de elemente din A
int nB=B.length;
int C[]=new int[nA+nB];
//Algoritmul de interclasare:
//Indexul initial in fiecare din cele 3 multimi:
//indexA=indexB=indexC=0 (se pleaca de la 0)
//Comparam pe A[ indexA] cu B[indexB].
//Pe cel mai mic, il copiem in multimea C.
//Modificam corespunzator indecsii: incrementam indexul multimii C
//si indexul din multimea din care a fost elementul ce s-a copiat.
//Se repeta apoi acest procedeu., pana se termina una din multimi.
int indexA=0;
int indexB=0;
int indexC=0;
int i;
while((indexA<nA)&&(indexB<nB)){
    if(A[indexA]<=B[indexB]){
        C[indexC]=A[indexA];
        indexA++;}
    else{
        C[indexC]=B[indexB];
        indexB++;}
    //in ambele cazuri, cresc indexC:
    indexC++;
}
//Una din multimile A , B - s-a terminat. Adaug la sfarsitul lui C
//restul elementelor din multimea ce nu s-a terminat:
if(indexA==nA)//s-a terminat A:
    for(i=indexB;i<nB;i++)//cele ramase in B:
        C[indexC+i-indexB]=B[i];
else if(indexB==nB)//s-a terminat B:
    for(i=indexA;i<nA;i++)//cele ramase in A:
        C[indexC+i-indexA]=A[i];
//Afisarea multimii A:
for(i=0;i<nA;i++)
    System.out.print(A[i]+" ");
System.out.println();
//Afisarea multimii B:
```

```
for(i=0;i<nB;i++)
    System.out.print(B[i]+" ");
System.out.println();
//Afisarea multimii C:
for(i=0;i<nA+nB;i++)
    System.out.print(C[i]+" ");
}
}
```

9. Folosind clasa Vector, implementați următorul algoritm:

se repetă

citim un număr real
îl memorăm într-un obiect de tip Vector
se afișează mesajul: "Dati un alt numar? [d/n] "
se citește răspunsul la acest mesaj

până când răspunsul este 'n'

Se vor afișa în final numerele memorate în obiectul Vector

```
import java.util.*;
import javax.swing.*;
class VectorDinamic
{
    public static void main(String args[])
    {
        Vector v=new Vector();
        double nr;
        for(;;){
            nr=Double.parseDouble(JOptionPane.showInputDialog("nr.="));
            v.add(new Double(nr));
            String raspuns=
                JOptionPane.showInputDialog("Dati un alt    numar? [d/n]: ");
            if(raspuns.charAt(0)=='n')break;
        }
        afisareVector(v);
        //Modificam primul numar introdus anterior in Vector:
        v.set(0,new Double(7.5));
        System.out.println("Vectorul modificat: ");
        afisareVector(v);
        System.out.println();
    }
    private static void afisareVector(Vector v)
```

```

{
    int n=v.size();// dimensiunea obiectului Vector:
    for(int i=0;i<n;i++){
        Double obiectCrt=(Double)v.get(i);
        //fara operatorul cast (Double): eroare, caci metoda get()
        //returneaza un obiect de tipul Object
        System.out.println(obiectCrt.doubleValue());
        //dintr-un obiect Double se extrage valoarea lui numerica de tipul
        //double, cu metoda doubleValue()
    }//for
}
}

```

10. Să se scrie o aplicație Java în care se implementează algoritmul de sortare rapidă (quicksort) a unui vector de numere întregi. Vectorul va fi inițializat cu N numere aleatoare.

```

import java.util.*;
class QuickSort
{
    public static void main(String args[])
    {
        final int GAMA=100;
        Random r=new Random();
        final int N=10;
        int a[]=new int[N];
        int i;
        for(i=0;i<N;i++)
            a[i]=r.nextInt(GAMA);
        System.out.println("Vector nesortat:");
        for(i=0;i<N;i++)
            System.out.println(a[i]);
        qsort(a,0,N-1);
        System.out.println("Vector sortat:");
        for(i=0;i<N;i++)
            System.out.println(a[i]);
    }

    private static void qsort(int a[],int st, int dr)
    {
        if(st<dr){

```

```
    int p=getPivot(a,st,dr);
    qsort(a,st,p-1);
    qsort(a,p+1,dr);
}
//cand st=dr se iese din metoda
}

private static int getPivot(int a[],int st, int dr)
{
    int i,j,di,dj;
    int aux;
    //Se parcurge vectorul din ambele capete:
    i=st;
    di=0;//pasul de avansare din capul stanga
    j=dr;
    dj=1;//pasul de avansare din capul dreapta
    while(i<j){
        if(a[i]>a[j]){ //daca nu sunt in ordine
            aux=a[i]; a[i]=a[j]; a[j]=aux; //comutare elemente
            //comutam si pasii:
            aux=di; di=dj; dj=aux;
        }//if
        i=i+di;
        j=j-dj;
    }//end while
    return i;
}
} //end class
```

11. Să se scrie o aplicație Java în care se implementează algoritmul de sortare bubblesort a unui vector de numere întregi.

```
class Sortare
{
    public static void main(String args[])
    {
        int a[]={1,7,6,5,10,4,3,2,9,8};
        int i;
        bubbleSort(a);
        //afisare:
        for(i=0;i<a.length;i++)
```

```

    System.out.println(a[i]);
}
private static void bubbleSort(int a[])
{
    int dim=a.length;
    int i;
    boolean suntInv;
    for(;;){
        suntInv=false;
        for(i=0;i<dim-1;i++){
            if(a[i]>a[i+1]){
                int aux=a[i];
                a[i]=a[i+1];
                a[i+1]=aux;
                suntInv=true;
            }
        }
        if(suntInv==false)break;
    }
}
}

```

12. Se citesc două mulțimi de numere întregi, (în fiecare mulțime, numerele sunt diferite), în doi vectori A și B. Să se calculeze reuniunea celor două mulțimi.

```

class Reuniune
{
    public static void main(String args[]){
        int A[]={1,10,2};
        int B[]={5, 6, 7, 2, 10};
        int nA=A.length;//nr. de elemente din A
        int nB=B.length;
        int i;
        int C[]=new int[nA+nB];//vectorul reuniune
        //Copiem pe A in C:
        for(i=0;i<nA;i++){
            C[i]=A[i];
        }
        //Adaugam in C[] toate elementele din B[], care
        // nu sunt si in A[]:
        int iC=nA;//index in multimea C
        for(i=0;i<nB;i++){

```

```

        if(estePrezent(B[i],A)==false){
            C[iC]=B[i];
            iC++;
        }
    int nC=iC;//numarul de elemente din C[]
    //Afisare A[]:
    for(i=0;i<nA;i++)
        System.out.print(A[i]+" ");
    System.out.println();
    //Afisarea multimii B:
    for(i=0;i<nB;i++)
        System.out.print(B[i]+" ");
    System.out.println();
    //Afisarea multimii C:
    for(i=0;i<nC;i++)
        System.out.print(C[i]+" ");
    }
    private static boolean estePrezent(int x,int a[])
    {
        for(int i=0;i<a.length;i++)
            if(a[i]==x)return true;
        return false;
    }
}

```

13. Se citește un număr natural N. Se citește un vector a de N numere întregi. Se citește un număr natural S. Să se afișeze toate posibilitățile de a scrie pe S ca sumă de numere luate din vectorul a (pot fi luate 1, 2, ...N numere din vector).

```

import javax.swing.*;
class ToatePosibilitatile
{
    private static int N;
    private static int S;
    private static int X[]=new int[100];//X[k] este din multimea {0,1}
    //X[i]=1 daca se ia in grupa pe a[i]. Daca nu se ia: X[i]=0 .
    //Vectorul X are N elemente (am dat o dimensiune acoperitoare)
    private static int a[]=new int[100];
    public static void main(String args[])
    {

```

```

S=Integer.parseInt(JOptionPane.showInputDialog("S="));
N=Integer.parseInt(JOptionPane.showInputDialog("N="));
int i;
for(i=0;i<N;i++)
    a[i]=Integer.parseInt(JOptionPane.showInputDialog("nr.="));
int nrElemente;
for(nrElemente=1;nrElemente<=N;nrElemente++)
    back(0,nrElemente);//generam toate grupele de nrElemente luate
//din multimea a[] (combinari)
}
private static void back(int k,int nrElemente)
{
    //construieste pe X[k]:
    int i;
    if(k==N)afisare();
    else for(i=0;i<=1;i++){
        X[k]=i;
        if(valid(k,nrElemente)==true)
            back(k+1,nrElemente);
    }
}
private static boolean valid(int k, int nrElemente)
{
    //Calculeaza cate elemente au fost luate si suma elementelor
// a[i] corespunzatoare:
    int contor=0;
    int suma=0;
    int i;
    for(i=0;i<=k;i++)
        if(X[i]==1){
            contor++;
            suma=suma+a[i];}
    if(contor>nrElemente)return false;
    if(suma>S)return false;
    if(k==N-1){//s-a completat vectorul X[]
        if(suma!=S)return false;
        if(contor!=nrElemente)return false;}
    return true;
}
private static void afisare()

```

```
{
    for(int i=0;i<N;i++)
        if(X[i]==1)System.out.print(a[i]+" ");
    System.out.println();
}
} //end class
```

14. Să se construiască clasa *Punct* ce are ca variabile de instanță două numere întregi *x* și *y* – coordonatele unui punct în plan, și ca metode:

- Constructorul ce face inițializările;
- Metoda *getX()* ce returnează valoarea coordonatei *x*
- Metoda *getY()* ce returnează valoarea coordonatei *y*
- Metoda *distanța()* ce are ca parametru un *Punct p* și care returnează distanța dintre punctul curent (referit prin cuvântul cheie *this*) și punctual *p*.
- Metoda *suntEgale()* ce are ca parametru un *Punct p* și care returnează *true* dacă punctul curent este egal cu *p*, și *false* în caz contrar.

Folosind clasa *Punct*, citim de la tastatură *N* obiecte *Punct* (*N* cunoscut), și le memorăm într-un vector. Să se afișeze:

- a. Sunt toate punctele diferite între ele?
- b. Câte puncte sunt situate pe prima bisectoare ($x=y$)?
- c. Care este cea mai mare distanță dintre două puncte din vector?

```
import javax.swing.*.*;
class Punct
{
    private int x;//coordonata x a punctului
    private int y;
    public Punct(int x0, int y0)
    {
        x=x0;
        y=y0;
    }
    public int getX()
    {
        return x;
    }
    public int getY()
    {
        return y;
    }
}
```

```

    }
    public double distanta(Punct p)
    {
        return Math.sqrt((x-p.x)*(x-p.x)+(y-p.y)*(y-p.y));
    }
    public boolean suntEgale(Punct p)
    {
        if((x==p.x)&&(y==p.y))return true;
        else return false;
    }
}

class VectorDePuncte
{
    public static void main (String args[])
    {
        int N;//numarul de puncte
        N=Integer.parseInt(JOptionPane.showInputDialog("N="));
        Punct p[]=new Punct[N];//vectorul de obiecte Punct (clasa de baza)
        int i;
        for(i=0;i<N;i++){
            int x=Integer.parseInt(JOptionPane.showInputDialog("x="));
            int y=Integer.parseInt(JOptionPane.showInputDialog("y="));
            p[i]=new Punct(x,y);
        }//for
        //Sunt toate punctele diferite?
        System.out.println("Sunt toate diferite="+suntDiferite(p));
        //Numarul de puncte de pe prima bisectoare:
        System.out.println("Nr. puncte pe prima
                                bisectoare="+catePeBisectoare(p));
        //Distanța maximă între două puncte:
        System.out.println("Distanța maximă="+distMaxima(p));
    }
    private static boolean suntDiferite(Punct p[])
    {
        for(int i=0;i<p.length-1;i++)
            for(int j=i+1;j<p.length;j++)
                if(p[i].suntEgale(p[j])!=true)return false;
        return true;
    }
}

```

```
private static int catePeBisectoare(Punct p[])
{
    int contor=0;
    for(int i=0;i<p.length;i++)
        if(p[i].getX()==p[i].getY())contor++;
    return contor;
}
private static double distMaxima(Punct p[])
{
    double maxim=0;//initializare distanta maxima
    //Formam toate perechile de puncte:
    for(int i=0;i<p.length-1;i++)
        for(int j=i+1;j<p.length;j++){
            double d=p[i].distanța(p[j]);
            if(d>maxim)maxim=d;
        }
    return maxim;
}
}
```

15. Să se construiască clasa Complex ce are ca variabile de instanțe private, numerele întregi re și im ce reprezintă partea reală și partea imaginară a unui număr complex. Ca metode:

- constructorul, ce face inițializările;
- metoda modul(), ce returnează modulul numărului complex;
- metoda suntEgale(), ce are ca parametru un număr complex c, și care returnează true dacă numărul complex curent (cel pentru care se apelează această metodă) este egal cu numărul complex c;
- metoda suma(), ce are ca parametru un număr complex c și în care se adună la numărul complex curent, numărul complex c (rezultatul se depune tot în numărul complex curent);
- metoda afisare(), în care se afișează numărul complex.

Folosind clasa Complex, citim de la tastatură, într-un vector, N numere complexe (N - cunoscut).

- a. să se afișeze care este numărul ce are cel mai mare modul;
- b. să se afișeze suma celor N numere complexe;
- c. să se afișeze dacă cele N numere complexe sunt toate diferite între ele.

```
import javax.swing.*;
class Complex
```

```
{
    private int re;//partea reala a numarului complex
    private int im;
    public Complex(int x, int y)
    {
        re=x;
        im=y;
    }
    public int getRe()
    {
        return re;
    }
    public int getIm()
    {
        return im;
    }
    public double modul()
    {
        return Math.sqrt(re*re+im*im);
    }
    public boolean suntEgale(Complex c)
    {
        if((re==c.re)&&(im==c.im))return true;
        else return false;
    }
    public void suma(Complex c)
    {
        re=re+c.re;
        im=im+c.im;
    }
}
```

```
class VectorDeComplexe
{
    public static void main (String args[])
    {
        int N;//numarul de numere complexe
        N=Integer.parseInt(JOptionPane.showInputDialog("N="));
        Complex c[]=new Complex[N];//vectorul de obiecte Complex
    }
}
```

```
int i;
for(i=0;i<N;i++){
    int x=Integer.parseInt(JOptionPane.showInputDialog("x="));
    int y=Integer.parseInt(JOptionPane.showInputDialog("y="));
    c[i]=new Complex(x,y);
}
//for
//Numarul ce are cel mai mare modul:
int index=calculIndexNrMaxModul(c);
System.out.println("Numarul ce are modulul maxim : ");
System.out.println("re="+c[index].getRe()+" im="
                    +c[index].getIm());

//Suma celor N numere complexe:
Complex suma=calculSuma(c);
System.out.println("suma este: re="+suma.getRe()+" im="
                    +suma.getIm());
//Sunt toate numerele complexe diferite?
System.out.println("Sunt toate diferite="+suntDiferite(c));
}
private static int calculIndexNrMaxModul(Complex c[])
{
    int indexMax=0;
    double modulMax=c[0].modul();
    for(int i=1;i<c.length;i++){
        double modulCrt=c[i].modul();
        if(modulCrt>modulMax){
            indexMax=i;
            modulMax=modulCrt;}
    }
    return indexMax;
}
private static Complex calculSuma(Complex c[])
{
    Complex rezultat=new Complex(0,0);
    for(int i=0;i<c.length;i++)
        rezultat.suma(c[i]);
    return rezultat;
}
private static boolean suntDiferite(Complex c[])
{
    for(int i=0;i<c.length-1;i++)
```

```

        for(int j=i+1;j<c.length;j++)
            if(c[i].suntEgale(c[j])!=true)return false;
        return true;
    }
}

```

16. Se citește de la tastatură o matrice de numere întregi, de dimensiuni cunoscute (numărul de linii și numărul de coloane citite anterior). Să se calculeze și afișeze dacă matricea are toate numerele egale între ele.

```

import javax.swing.*;
class MatriceSuntEgale
{
    public static void main(String args[])
    {
        int nL;//numar linii matrice
        nL=Integer.parseInt(JOptionPane.showInputDialog("nr. linii="));
        int nC;//numar coloane matrice
        nC=Integer.parseInt(
            JOptionPane.showInputDialog("nr. coloane="));
        int a[][]=new int[nL][nC];
        //citire matrice:
        int i,j;
        for(i=0;i<nL;i++)
            for(j=0;j<nC;j++)
                a[i][j]=Integer.parseInt(JOptionPane.showInputDialog("nr="));
        boolean egale=suntToateEgale(a);
        if(egale==true)System.out.println("Sunt toate numerele egale.");
        else System.out.println("Nu sunt toate egale");
    }
    private static boolean suntToateEgale(int a[][])
    {
        for(int i=0;i<a.length;i++)
            for(int j=0;j<a[0].length;j++)
                if(a[i][j]!=a[0][0])return false;
        return true;
    }
}

```

17. Se dă o matrice de numere întregi. Să se calculeze și afișeze dacă în această matrice există cel puțin două linii egale între ele.

```
class MatriceLiniiEgale
{
    public static void main(String args[])
    {
        int a[][]={{1,2,3,4,5},
                    {0,1,0,0,0},
                    {1,2,3,4,5},
                    {1,1,1,1,1}};
        //Comparam liniile matricii, doua cate doua:
        int nL=a.length;//numarul de linii
        int nC=a[0].length;//numarul de coloane
        int i,j;
        for(i=0;i<nL-1;i++)
            for(j=i+1;j<nL;j++)
                //compara linia i cu linia j:
                if(suntEgale(a[i],a[j])){
                    System.out.println("Are linii egale intre ele");
                    return;
                }
        System.out.println("Nu are linii egale intre ele")    ;
    }
    private static boolean suntEgale(int linie1[], int linie2[])
    {
        //Se compara doi vectori pentru egalitatea lor:
        for(int i=0;i<linie1.length;i++)
            if(linie1[i]!=linie2[i])return false;
        return true;
    }
}
```

18. Se dă o matrice de numere întregi.. Să se calculeze și afișeze dacă în această matrice există cel puțin o linie care să aibă toate elementele egale între ele.

```
class MatriceLinieConstanta
{
    public static void main(String args[])
    {
        int a[][]={{1,2,3,4,5},
```

```

        {0,0,0,0,0},
        {1,2,3,4,5},
        {1,1,1,0,1}};
//Parcurgem toate liniile matricii:
int nL=a.length;//numarul de linii
for(int i=0;i<nL;i++)
    //are linia curenta i, toate elementele egale?
    if(areToateEgale(a[i])){
        System.out.println("Are.");
        return;
    }
    System.out.println("Nu are.");
}
private static boolean areToateEgale(int linie[])
{
    //Se compara toate elementele vectorului, cu primul:
    for(int i=1;i<linie.length;i++)
        if(linie[i]!=linie[0])return false;
    return true;
}
}

```

19. Se citește un număr natural N. Se inițializează o matrice pătrată de dimensiune N, cu numere aleatoare în gama 1...100. Să se calculeze câte numere prime sunt în matrice.

```

import java.util.*;
import javax.swing.*;
class CatePrimeInMatrice
{
    public static void main(String args[])
    {
        final int GAMA=100;
        int N=Integer.parseInt(JOptionPane.showInputDialog("N="));
        int a[][]=new int[N][N];
        Random r=new Random();
        //Initializam matricea:
        int i,j;
        for(i=0;i<N;i++)
            for(j=0;j<N;j++)
                a[i][j]=1+r.nextInt(GAMA);
        //Afisarea matricii:
    }
}

```



```
for(i=0;i<N;i++){
    for(j=0;j<N;j++){
        System.out.print(a[i][j]+" ");
        System.out.println();
    }
    int totalNrPrime=cateNrPrime(a);
    System.out.println("numarul de nr. prime = "+totalNrPrime);
}
private static int cateNrPrime(int a[][])
{
    int contor=0;
    for(int i=0;i<a.length;i++)
        for(int j=0;j<a[0].length;j++)
            if(estePrim(a[i][j]))contor++;
    return contor;
}
private static boolean estePrim(int nr)
{
    for(int i=2;i<=Math.sqrt(nr);i++)
        if(nr%i==0)return false;
    return true;
}
}
```

20. Se citește un număr natural N. Se inițializează o matrice pătrată de dimensiune N, cu numere aleatoare în gama 1...100. Să se afișeze dacă toate numerele din matrice sunt diferite între ele sau nu.

```
import java.util.*;
import javax.swing.*;
class MatriceSuntDiferite
{
    public static void main(String args[])
    {
        final int GAMA=100;
        int N=Integer.parseInt(JOptionPane.showInputDialog("N="));
        int a[][]=new int[N][N];
        Random r=new Random();
        //Initializam matricea:
        int i,j;
        for(i=0;i<N;i++)
            for(j=0;j<N;j++)
```

```

        a[i][j]=1+r.nextInt(GAMA);
//Afisarea matricii:
for(i=0;i<N;i++){
    for(j=0;j<N;j++)
        System.out.print(a[i][j]+" ");
    System.out.println();
}
boolean sunt=suntDiferiteInMatrice(a);
System.out.println("sunt diferite = "+sunt);
}
private static boolean suntDiferiteInMatrice(int a[][])
{
    int nL=a.length;//numar linii
    int nC=a[0].length;//numar coloane (in aceasta problema nC=nL)
    int i,j,lin,col;
    //Are prima linie toate elem. diferite?
    if(suntDiferiteInVector(a[0])!=false)return false;
    //Prima linie are toate elementele diferite.
    //Parcure restul liniilor:
    for(i=1;i<nL;i++){
        //are linia i toate elementele diferite intre ele?
        if(suntDiferiteInVector(a[i])!=false)return false;
        //Elem. liniei i mai apar in liniile 0,1,i-1?
        for(j=0;j<nC;j++){
            //Parcure liniile 0,1,...,i-1:
            for(lin=0;lin<i;lin++){
                for(col=0;col<nC;col++){
                    if(a[i][j]==a[lin][col])return false;
                }
            }
        }
        return true;
    }
}
private static boolean suntDiferiteInVector(int v[])
{
    for(int i=0;i<v.length-1;i++)
        for(int j=i+1;j<v.length;j++)
            if(v[i]==v[j])return false;
    return true;
}
}

```

21. Se citește de la tastatură o matrice de numere întregi, de dimensiuni cunoscute (numărul de linii și numărul de coloane citite anterior). Se citesc în variabilele l1 și l2, numerele a două linii din matrice. Să se comute în matrice aceste linii.

```
import javax.swing.*;
class ComutaDouaColoane
{
    public static void main(String args[])
    {
        int nL;//numar linii matrice
        nL=Integer.parseInt(JOptionPane.showInputDialog("nr. linii="));
        int nC;//numar coloane matrice
        nC=Integer.parseInt(JOptionPane.showInputDialog("nr. col.="));
        int a[][]=new int[nL][nC];
        //citire matrice:
        int i,j;
        for(i=0;i<nL;i++)
            for(j=0;j<nC;j++)
                a[i][j]=Integer.parseInt(JOptionPane.showInputDialog("nr="));
        //Citeste numerele celor doua linii:
        int l1;
        l1=Integer.parseInt(JOptionPane.showInputDialog("l1="));
        int l2;
        l2=Integer.parseInt(JOptionPane.showInputDialog("l2="));
        comutare(a,l1,l2);
        //Afisare matrice dupa comutare:
        System.out.println();
        for(i=0;i<nL;i++)
        {
            for(j=0;j<nC;j++)
                System.out.print(a[i][j]+" ");
            System.out.println();
        }
    }
    public static void comutare(int a[][],int l1,int l2)
    {
        int aux;
        int i;
        for (i=0;i<a[0].length;i++)
        {
```

```

    aux=a[l1][i];
    a[l1][i]=a[l2][i];
    a[l2][i]=aux;
}
}
}

```

22. Se citește un număr natural N , impar. Să se construiască o matrice pătrat magic de ordinul N , după următorul algoritm:

-Punem primul număr (1) în matrice, la mijlocul ultimei linii. (deci poziția inițială este: linia= $N-1$, coloana= $N/2$)

-Punem în ordine numerele $2, 3, \dots, N \cdot N$ în matrice, calculând astfel poziția următoare, în care vom scrie numărul la care am ajuns:

Poziția următoare: dreapta jos cu o linie, față de poziția curentă.

Dacă această nouă poziție a trecut de ultima linie, trecem pe prima linie (linia 0).

Dacă a trecut de ultima coloană, trecem pe coloana 0,

Dacă această nouă poziție calculată este deja ocupată, ne deplasăm în celula de deasupra celei în care s-a introdus numărul anterior (aceasta sigur nu este ocupată)

```
import javax.swing.*;
```

```
class ConstruieștePatratMagic
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        int N;
```

```
        for(;;){
```

```
            N=Integer.parseInt(JOptionPane.showInputDialog
                                ("N (impar)= "));
```

```
            if(N%2==1)break;
```

```
        }
```

```
        int a[][]=new int[N][N];
```

```
        int i,j,linCrt,colCrt;
```

```
//Umplem matricea cu 0: (nici un numar din gama 1..N*N nu a fost
//introdus)
```

```
        for(i=0;i<N;i++)
```

```
            for(j=0;j<N;j++)
```

```
                a[i][j]=0;
```

```
//Punem primul numar (1) in matrice, la mijlocul ultimei linii:
```

```
        linCrt=N-1;
```

```
        colCrt=N/2;
```

```
a[linCrt][colCrt]=1;
//Punem in ordine numerele 2,3,...,N*N in matrice, calculand
//conform algoritmului pozitiile pe care le punem in matrice:
for(int nr=2; nr<=N*N;nr++)
{
    //Pozitia curenta este coltul dreapta jos?
    if((linCrt==N-1)&&(colCrt==N-1))linCrt=N-2;//trecem pe
    //penultima linie
    else{
        //Calculam pozitia pe care scriem pe nr:
        //Salvam linia si coloana curenta (pentru cazul in care pozitia
        //ce o vom calcula, este deja ocupata):
        int linAnt=linCrt;
        int colAnt=colCrt;
        //Pozitia urmatoare: dreapta jos cu o linie, fata de pozitia
        //curenta:
        linCrt=linCrt+1;
        colCrt=colCrt+1;
        //Noua pozitie a depasit granitele matricii?
        if (linCrt==N)linCrt=0;//prima linie
        if (colCrt==N)colCrt=0;//prima coloana
        //Pozitia calculata este ocupata?
        if(a[linCrt][colCrt]!=0){
            //Este ocupata. Ne deplasam in celula de deasupra celei
            //in care s-a introdus numarul anterior:
            //(aceasta sigur nu este ocupata)
            linCrt=linAnt-1;
            colCrt=colAnt;}
        }//else
        a[linCrt][colCrt]=nr;
    }//for
    //Afisarea matricii patrat magic:
    for(i=0;i<N;i++){
        for(j=0;j<N;j++){
            System.out.print(a[i][j]+" ");
            System.out.println();
        }
    }
}
```

Cap. 4

Șiruri de caractere

1. Se citește un șir. Să se afișeze dacă primul caracter este egal cu ultimul caracter.

```
import javax.swing.*;
class PrimUltimEgale
{
    public static void main(String args[])
    {
        String s=JOptionPane.showInputDialog("sir = ");
        if(s.charAt(0)==s.charAt(s.length()-1))
            System.out.println("Primul caracter este egal cu ultimul caracter");
        else System.out.println
            ("Primul caracter este diferit de ultimul caracter");
    }
}
```

2. Se citesc de la tastatură sub formă de șiruri de caractere doi timpi, în formatul hh:mm:ss (ore: minute: secunde). Să se afișeze care timp este mai mare.

Exemplu:

T1= 5:35:42

T2= 5:18:50

Se va afișa: T2 > T1

```
import javax.swing.*;
import java.util.*;
class ComparaTimpi
{
    public static void main(String args[])
    {
        String timp1=JOptionPane.showInputDialog
            ("timp1 (hh:mm:ss) = ");
        String timp2=JOptionPane.showInputDialog
            ("timp2 (hh:mm:ss) = ");
        //Extragem din fiecare timp, orele, minutele si secunde.
        // Folosim clasa StringTokenizer pentru a extrage acesti atomi.
        StringTokenizer tk=new StringTokenizer(timp1,".");
```

```
int ore1=Integer.parseInt(tk.nextToken());
int min1=Integer.parseInt(tk.nextToken());
int sec1=Integer.parseInt(tk.nextToken());
//Calculam primul timp, in secunde:
int T1=3600*ore1+60*min1+sec1;
//Similar, pentru timp2 :
tk=new StringTokenizer(timp2,":");
int ore2=Integer.parseInt(tk.nextToken());
int min2=Integer.parseInt(tk.nextToken());
int sec2=Integer.parseInt(tk.nextToken());
int T2=3600*ore2+60*min2+sec2;
if(T1>T2)System.out.println("timp1 > timp2");
else if(T1==T2)System.out.println("timp1 = timp2");
else System.out.println("timp1 < timp2");
}
}
```

3. Se citește un șir s1 de la tastatură, să se construiască un alt șir s2: inversul șirului s1.

```
import javax.swing.*;
class InvString
{
    public static void main(String args[])
    {
        String s1=JOptionPane.showInputDialog("sir=");
        String s2="";
        int i;
        int L1=s1.length();//lungimea primului sir
        for(i=L1-1;i>=0;i--)
            s2=s2+s1.charAt(i);
        System.out.println("Inversul sirului "+s1+" este "+s2);
    }
}
```

4. Se citesc două șiruri s1 și s2. Să se afișeze care șir conține mai multe vocale.

```
import javax.swing.*;
class ComparaDupaVocale
{
    public static void main(String args[])
    {
```

```

{
    String s1=JOptionPane.showInputDialog("sir1 = ");
    String s2=JOptionPane.showInputDialog("sir2 = ");
    int nv1=numarVocale(s1);
    int nv2=numarVocale(s2);
    if(nv1>nv2)System.out.println("Sir 1 are mai multe vocale ca sir 2");
    else if(nv1==nv2)System.out.println("numar egal de vocale");
    else System.out.println("Sir 2 are mai multe vocale ca sir 1");;
}
private static int numarVocale(String s)
{
    int contor=0;
    for(int i=0;i<s.length();i++)
        if(esteVocala(s.charAt(i))==true)contor++;
    return contor;
}
private static boolean esteVocala(char ch)
{
    if((ch=='a')||(ch=='A')||
        (ch=='e')||(ch=='E')||
        (ch=='i')||(ch=='I')||
        (ch=='o')||(ch=='O')||
        (ch=='u')||(ch=='U'))return true;
    else return false;
}
}

```

5. Se citește un șir. Să se afișeze dacă are toate caracterele egale între ele. Se va folosi o metodă separată: areToateCarEgale().

```

import javax.swing.*;
class ToateCarEgale
{
    public static void main(String args[])
    {
        String s=JOptionPane.showInputDialog("sir = ");
        if(areToateCarEgale(s)==true)
            System.out.println("Are toate caracterele egale");
        else System.out.println("Nu are toate caracterele egale");
    }
    private static boolean areToateCarEgale(String s)

```

```
{  
    //Le comparăm pe toate, cu primul caracter:  
    for(int i=1;i<s.length();i++)  
        if(s.charAt(i)!=s.charAt(0))return false;  
    return true;  
}  
}
```

6. Se citește un șir. Să se afișeze dacă are toate caracterele diferite între ele. Se va folosi o metodă separată: suntToateDiferite().

```
import javax.swing.*;  
class ToateCarDiferite  
{  
    public static void main(String args[])  
    {  
        String s=JOptionPane.showInputDialog("sir = ");  
        if(suntToateDiferite(s)==true)  
            System.out.println("Are toate caracterele diferite");  
        else System.out.println("Nu are toate caracterele diferite");  
    }  
    private static boolean suntToateDiferite(String s)  
    {  
        //Comparăm caracterul curent, cu toate caracterele de după el:  
        for(int i=0;i<s.length()-1;i++)  
            for(int j=i+1;j<s.length();j++)  
                if(s.charAt(i)==s.charAt(j))return false;  
        return true;  
    }  
}
```

7. Se citește un șir. Să se afișeze dacă șirul conține numai vocale. Se va folosi o metodă separată: suntNumaiVocale().

```
import javax.swing.*;  
class SuntNumaiVocale  
{  
    public static void main(String args[])  
    {  
        String s=JOptionPane.showInputDialog("sir = ");  
        System.out.println("are numai vocale = "+suntNumaiVocale(s));  
    }  
}
```

```

private static boolean suntNumaiVocale(String s)
{
    for(int i=0;i<s.length();i++)
        if(esteVocala(s.charAt(i))!=false)return false;
    return true;
}
private static boolean esteVocala(char ch)
{
    if((ch=='a')||(ch=='A')||
        (ch=='e')||(ch=='E')||
        (ch=='i')||(ch=='I')||
        (ch=='o')||(ch=='O')||
        (ch=='u')||(ch=='U'))return true;
    else return false;
}
}

```

8. Se citește un șir s1 de la tastatură. Să se construiască un alt șir s2, obținut prin criptarea șirului inițial după algoritmul lui Cezar. (La început se convertesc toate literele mari din șirul s1, în litere mici. Apoi fiecare literă mică din s1 se criptează în șirul s2 astfel: a se înlocuiește cu d, b cu e,..., x cu a, y cu b și z cu c; se observă că distanța dintre literă și litera criptată este 3).

```

import javax.swing.*;
class CriptareString
{
    public static void main(String args[])
    {
        String s=JOptionPane.showInputDialog("sir = ");
        s=s.toLowerCase();
        String sCriptat="";
        for(int i=0;i<s.length();i++)
            if(esteLitera(s.charAt(i)))
                sCriptat=sCriptat+literaCriptata(s.charAt(i));
            else sCriptat=sCriptat+s.charAt(i);
        System.out.println("sir criptat = "+sCriptat);
    }
    private static boolean esteLitera(char ch)
    {
        int cod=(int)ch;
        if((cod>=(int)'a')&&(cod<=(int)'z'))return true;
    }
}

```

```
    else return false;
}
private static char literaCriptata(char ch)
{
    if(ch=='x')return 'a';
    if(ch=='y')return 'b';
    if(ch=='z')return 'c';
    //Pentru restul literelor:
    int cod=(int)ch;
    int noulCod=cod+3;
    return (char)noulCod;
}
}
```

9. Se citește un număr natural N. Se citesc de la tastatură N cuvinte, într-un vector de șiruri. Să se afișeze dacă toate cuvintele sunt diferite între ele sau nu.

```
import javax.swing.*;
class VectorCuvinteDiferite
{
    public static void main(String args[])
    {
        int N=Integer.parseInt(JOptionPane.showInputDialog("N = "));
        String s[]=new String[N];
        int i,j;
        for(i=0;i<N;i++)
            s[i]=JOptionPane.showInputDialog("cuvant = ");
        //Sunt diferite?
        for(i=0;i<N-1;i++)
            for(j=i+1;j<N;j++)
                if(s[i].compareTo(s[j])==0){
                    System.out.println("Nu sunt toate diferite . ");
                    return;
                }
        System.out.println("Sunt toate diferite ");
    }
}
```

10. Se citește un număr natural N. Se citesc de la tastatură N cuvinte, într-un vector de șiruri. Să se afișeze cuvântul de lungime maximă.

```
import javax.swing.*;
class VectorCuvinteMaxim
{
    public static void main(String args[])
    {
        int N=Integer.parseInt(JOptionPane.showInputDialog("N = "));
        String s[]=new String[N];
        int i,j;
        for(i=0;i<N;i++)
            s[i]=JOptionPane.showInputDialog("cuvant = ");
        //Initializare:
        int lungMax=s[0].length();//lungimea maxima
        int indexMax=0;
        //Parcurgem restul vectorului:
        for(i=1;i<N;i++)
            if(s[i].length()>lungMax){
                lungMax=s[i].length();
                indexMax=i;
            }
        System.out.println("Cuvantul de lungime maxima este: "
            +s[indexMax]);
    }
}
```

11. Se citește un număr natural N. Se citesc de la tastatură N cuvinte, într-un vector de șiruri. Să se sorteze în ordine alfabetică.

```
import javax.swing.*;
class VectorCuvinteSortare
{
    public static void main(String args[])
    {
        int N=Integer.parseInt(JOptionPane.showInputDialog("N = "));
        String s[]=new String[N];
        int i,j;
        for(i=0;i<N;i++)
            s[i]=JOptionPane.showInputDialog("cuvant = ");
        //Sortarea prin interschimbare:
```

```
for(i=0;i<N-1;i++)
    for(j=i+1;j<N;j++)
        if(s[i].compareTo(s[j])>0){
            //comut pe s[i] cu s[j]:
            String aux=s[i];
            s[i]=s[j];
            s[j]=aux;
        }
//Afisare vector sortat:
for(i=0;i<N;i++)
    System.out.println(s[i]);
}
```

Cap. 5

Clase derivate. Clase abstracte. Polimorfism

1. Să se construiască clasa *ContBancar*, folosită pentru a modela un cont bancar, ce are ca variabilă de instanță privată, variabila *suma*, (suma de bani din cont). Ca metode:

- constructorul;
- *adauga()*, ce are ca parametru un număr real *x*, valoarea ce se adaugă în cont;
- *extrage()*, ce are ca parametru un număr real *x*, valoarea ce se extrage din cont, și care scoate ca rezultat *true*, dacă se poate face extragerea ($\text{suma} \geq x$), și *false* în caz contrar;
- *getSuma()*, ce returnează valoarea variabilei private *suma*;
- *afisare()*, ce afișează valoarea sumei de bani din cont.

Pe baza clasei *ContBancar* se va dezvolta prin derivare (moștenire) clasa *ContBancarExtins*, în care se va adăuga o nouă variabilă de instanță: *rata dobânzii anuale* și o nouă metodă: *adaugaDobandaLunara()*, ce adaugă în cont dobânda calculată după trecerea unei luni. În clasa *ContBancarExtins* se va redefini și metoda *afisare()*, astfel încât să se afișeze și *rata dobânzii*. De asemenea, în această nouă clasă se va defini constructorul, prin care se inițializează *suma* de bani din cont și *rata dobânzii*.

Să se scrie și o clasă de test pentru clasa *ContBancarExtins*.

```
class ContBancar
{
    private double suma;
    public ContBancar(double S)
    {
        suma=S;
    }
    public void adauga(double S)
    {
        suma=suma+S;
    }
    public boolean extrage(double S)
    {
        if(S>suma)return false;
        suma=suma-S;
        return true;
    }
}
```

```
}
public double getSuma()
{
    return suma;
}
public void afisare()
{
    System.out.println("suma="+suma);
}
}

class ContBancarExtins extends ContBancar
{
    private double rd;//rata dobanzii anuale
    public ContBancarExtins(double S,double rata)
    {
        super(S);
        rd=rata;
    }
    public void adaugaDobandaLunara()
    {
        double S=this.getSuma();
        double dobanda=S*rd/12;
        this.adauga(dobanda);
    }
    public void afisare()
    {
        System.out.println("suma="+this.getSuma());
        System.out.println("rata dobanzii="+rd);
    }
}

class TestCont
{
    public static void main(String args[])
    {
        ContBancarExtins c=new ContBancarExtins(1000,0.12);
        c.adauga(1000);
        c.adaugaDobandaLunara();
        c.afisare();
    }
}
```

```
}
}
```

2. Să se construiască clasa `Punct3D`, folosită pentru a modela un punct în spațiu, ce are ca variabile de instanță `x`, `y`, `z`, coordonatele unui punct în spațiu. Ca metode:

- constructorul;
- `muta()`, ce are trei parametri `dx`, `dy` și `dz`, pe baza cărora noile coordonate ale punctului devin: `x+dx`, `y+dy`, `z+dz`;
- `compara()`, ce are ca parametru un punct `p`, și care returnează *true*, dacă punctul curent (cel referit prin *this*) este egal cu punctul `p`, și *false* în caz contrar;
- `distanța()`, ce are ca parametru un punct `p`, și care returnează distanța între punctul curent și punctul `p`;
- `getX()` ce returnează valoarea coordonatei `x`;
- `getY()` ce returnează valoarea coordonatei `y`;
- `getZ()` ce returnează valoarea coordonatei `z`;
- `afisare()` ce afișează coordonatele punctului.

Pe baza clasei `Punct3D`, se va dezvolta clasa `Punct3DColor`, în care se va adăuga o nouă variabilă de instanță de tipul `String`: culoarea punctului și o nouă metodă `getCuloare()` ce returnează culoarea punctului. Se vor redefini metodele `compara()` și `afisare()` și noul constructor.

Să se scrie și o clasă de test pentru clasa `Punct3DColor`.

```
class Punct3D
{
    private int x;//coordonata x a punctului
    private int y;
    private int z;
    public Punct3D(int x,int y, int z )
    {
        this.x=x;
        this.y=y;
        this.z=z;
    }
    public int getX()
    {
        return x;
    }
    public int getY()
```



```
{
    return y;
}
public int getZ()
{
    return z;
}
public void afisare()
{
    System.out.println("x="+x);
    System.out.println("y="+y);
    System.out.println("z="+z);
}
public void muta(int dx, int dy, int dz)
{
    x=x+dx;
    y=y+dy;
    z=z+dz;
}
public boolean compara(Punct3D p)
{
    if((x==p.x)&&(y==p.y)&&(z==p.z))
        return true;
    else return false;
}
public double distanta(Punct3D p)
{
    double dx=this.x-p.x;
    double dy=this.y-p.y;
    double dz=this.z-p.z;
    double dist=Math.sqrt(dx*dx+dy*dy+dz*dz);
    return dist;
}
}
class Punct3DColor extends Punct3D
{
    private String culoare;
    public Punct3DColor(int x, int y, int z, String culoare)
    {
        super(x,y,z);
    }
}
```

```
    this.culoare=culoare;
}
public String getCuloare()
{
    return culoare;
}

public void afisare()
{
    System.out.println("x="+getX());
    System.out.println("y="+getY());
    System.out.println("z="+getZ());
    System.out.println("culoare="+culoare);
}
public boolean compara(Punct3DColor p)
{
    if((this.getX()==p.getX())&&
        (this.getY()==p.getY())&&
        (this.getZ()==p.getZ())&&
        (this.culoare==p.culoare))
        return true;
    else return false;
}
}

class TestPuncte
{
    public static void main (String args[])
    {
        Punct3DColor p=new Punct3DColor(0,1,2,"negru");
        p.muta(1,1,1);
        p.afisare();
    }
}
```

3. Să se dezvolte clasa abstractă A în care sunt definite două metode:

- metoda abstractă calcul();
 - metoda durataCalcul() ce returnează durata exprimată în milisecunde, a execuției metodei calcul();
-

Din clasa abstractă A, se va deriva clasa B ce conține implementarea metodei calcul(). Se va dezvolta și o clasă de test, pentru clasa derivată B.

```
import javax.swing.*;
import java.util.*;
abstract class A
{
    abstract public void calcul(int N);
    public long durataCalcul(int N){
        long t1=System.currentTimeMillis();
        calcul(N);
        long t2=System.currentTimeMillis();
        return (t2-t1);
    }
}
class B extends A
{
    public void calcul(int N)
    {
        //Calculeaza N*N*N produse
        int i,j,k;
        long rezultat;
        for(i=1;i<=N;i++)
            for(j=1;j<=N;j++)
                for(k=1;k<=N;k++)
                    rezultat=i*j*k;
    }
}
class Test
{
    public static void main(String args[])
    {
        final int N=1000;
        B b=new B();
        System.out.println("durata calcul = "+b.durataCalcul(N)+" ms.");
    }
}
```

4. Să se construiască clasa `Punct` ce are ca variabile de instanță două numere întregi `x` și `y` – coordonatele unui punct în plan, și ca metode:

- Constructorul ce face inițializările;
- `getX()` ce returnează valoarea coordonatei `x`
- `getY()` ce returnează valoarea coordonatei `y`
- `afisare()` în care se afișează coordonatele punctului din clasa `Punct`

Din clasa `Punct` se derivează două clase: `PunctColor` și `Punct3D`.

Clasa `PunctColor` față de clasa `Punct` are în plus o variabilă de instanță în care este memorată culoarea punctului, un nou constructor în care este inițializată și culoarea, metoda `getCuloare()` ce returnează culoarea, și redefinește metoda clasei de bază, `afisare()`, afișând pe lângă coordonatele `x` și `y` și culoarea.

Clasa `Punct3D`, ce reprezintă un punct în spațiu, față de clasa `Punct` are în plus o variabilă de instanță `z`, un nou constructor în care sunt inițializate toate cele trei coordonate, metoda `getZ()` ce returnează valoarea coordonatei `z`, și redefinește metoda clasei de bază, `afisare()`, afișând pe lângă coordonatele `x` și `y` și coordonata `z`.

Folosind aceste trei clase se va dezvolta o aplicație în care se vor citi de la tastatură `N` puncte (`N`- citit anterior), de tipul `PunctColor` sau `Punct3D`. Pentru fiecare punct, în momentul citirii utilizatorul aplicației va specifica dacă va introduce un `PunctColor` sau un `Punct3D`. Cele `N` puncte se vor memora într-un vector de obiecte de tipul `Punct` (clasa de bază). În final se vor afișa pentru fiecare punct din cele `N` informațiile memorate (pentru fiecare punct se va apela metoda `afisare()`). Această aplicație ilustrează conceptul de polimorfism (Compilerul știe la rulare ce versiune de metodă `afisare()` să apeleze).

```
import javax.swing.*;
```

```
class Punct
```

```
{
    private int x;//coordonata x a punctului
    private int y;
    public Punct(int x0, int y0)
    {
        x=x0;
        y=y0;
    }
    public int getX()
```

```
{
    return x;
}
public int getY()
{
    return y;
}
public void afisare()
{
    System.out.println("x="+x);
    System.out.println("y="+y);
}
}
```

```
class PunctColor extends Punct
{
    private String culoare;
    public PunctColor(int x, int y, String culoare)
    {
        super(x,y);
        this.culoare=culoare;
    }
    public String getCuloare()
    {
        return culoare;
    }
}
```

```
public void afisare()
{
    //System.out.println("x="+x);GRESIT! x - este var. privata !
    System.out.println("x="+getX());
    System.out.println("y="+getY());
    System.out.println("culoare="+culoare);
}
}
```

```
class Punct3D extends Punct
{
    private int z;
    public Punct3D(int x,int y, int z )
    {
        super(x,y);
        this.z=z;
    }
}
```

```

{
    super(x,y);
    this.z=z;
}
public int getZ()
{
    return z;
}
public void afisare()
{
    System.out.println("x="+getX());
    System.out.println("y="+getY());
    System.out.println("z="+z);
}
}

```

```

class AfisarePuncte
{
    public static void main (String args[])
    {
        int N;//numarul de puncte
        N=Integer.parseInt(JOptionPane.showInputDialog("N="));
        Punct p[]=new Punct[N];//vectorul de obiecte Punct (clasa de
        //baza)
        int i;
        for(i=0;i<N;i++){
            String sRaspuns=JOptionPane.showInputDialog("Tip punct
            (0,1,2) : ");
            int raspuns=Integer.parseInt(sRaspuns);
            int x=Integer.parseInt(JOptionPane.showInputDialog("x="));
            int y=Integer.parseInt(JOptionPane.showInputDialog("y="));
            if(raspuns==0){
                //citeste un Punct:
                p[i]=new Punct(x,y);}
            else if(raspuns==1){
                //citeste un PunctColor:
                String culoare=JOptionPane.showInputDialog("culoare=");
                p[i]=new PunctColor(x,y,culoare);}
            else if(raspuns==2){
                //citeste un Punct3D:

```

```
        int z=Integer.parseInt(JOptionPane.showInputDialog("z="));
        p[i]=new Punct3D(x,y,z);}
    }//for
    //Afisare vector:
    for(i=0;i<N;i++)
        p[i].afisare();
    }
}
```

5. Să se dezvolte clasa Persoana ce are ca variabile de instanță numele și prenumele unei persoane și vârsta ei, și ca metode:

- constructorul ce face inițializările;
- getNume(), ce returnează numele;
- getPrenume(), ce returnează prenumele;
- afisare(), ce afișează informațiile despre persoană.

Din clasa Persoană se va deriva clasa Student, ce are în plus ca variabile de instanță, numele facultății pe care o urmează și numărul matricol. În clasa Student se va dezvolta un nou constructor și se va redefini metoda afisare(). Se vor adăuga în plus metodele:

- getFacultate();
- getNumărMatricol().

Se va dezvolta o aplicație în care se vor citi de la tastatură N= 10 studenți, ce se vor memora într-un vector. Se vor afișa câți studenți au prenumele "Ion".

```
import javax.swing.*.*;
class Persoana
{
    private String nume;
    private String prenume;
    private int varsta;
    public Persoana(String n, String p, int v)
    {
        nume=n;
        prenume=p;
        varsta=v;
    }
    public String getNume()
    {
        return nume;
    }
}
```

```
public String getPrenume()
{
    return prenume;
}
public void afisare()
{
    System.out.println(ume+" "+prenume+" : "+varsta);
}
}
class Student extends Persoana
{
    private String numeFacultate;
    private int nrMatricol;
    public Student(String n, String p, int v, String facult, int nrMatr)
    {
        super(n,p,v);
        numeFacultate=facult;
        nrMatricol=nrMatr;
    }
    public String getFacultate()
    {
        return numeFacultate;
    }
    public int getNumarMatricol()
    {
        return nrMatricol;
    }
}
class TestStudenti
{
    public static void main(String args[])
    {
        final int N=2;
        int i;
        Student s[]=new Student[N];
        for(i=0;i<N;i++){
            String nume=JOptionPane.showInputDialog("nume=");
            String prenume=JOptionPane.showInputDialog("prenume=");
            int varsta=
                Integer.parseInt(JOptionPane.showInputDialog("varsta="));
```

```
String facultate=JOptionPane.showInputDialog("facultate=");
int nrMatr=Integer.parseInt
(JOptionPane.showInputDialog("nr. matricol="));
s[i]=new Student(nume,prenume,varsta,facultate,nrMatr);
}
int contor_ion=0;
for(i=0;i<N;i++){
String prenumeCrt=s[i].getPrenume();
if(prenumeCrt.compareTo("Ion")==0)contor_ion++;
}
System.out.println(contor_ion);
}
}
```

6. Să construiască clasa Număr ce are ca variabilă de instanță un număr întreg a, și ca metode:

- constructorul;
- afisare().

Din clasa Numar, se va deriva clasa DouaNumere, în care se va adăuga variabila de instanță b (ce reprezintă al doilea număr), și se va modifica constructorul și metoda afisare().

Folosind cele două clase, se va dezvolta o aplicație în care se generează un număr aleator 0 sau 1. Dacă este 0, se va instanția un obiect din clasa Numar (prin citire de la tastatură), dacă este 1, se va instanția un obiect din clasa DouaNumere (tot prin citire de la tastatură). Pentru obiectul instanțiat se va apela metoda afisare().

```
import java.util.*;
import javax.swing.*;
class Numar
{
private int a;
public Numar(int x)
{
a=x;
}
public void afisare()
{
System.out.println("a="+a);
}
}
```

```

class DouaNumere extends Numar
{
    private int b;
    public DouaNumere(int a,int b )
    {
        super(a);
        this.b=b;
    }
    public void afisare()
    {
        //afisare a:
        super.afisare();//Cu cuv. cheie super se apeleaza metoda
                        //clasei de baza
        //afisare b:
        System.out.println("b="+b);
    }
}

class AfisareNumere
{
    public static void main (String args[])
    {
        Numar n;
        Random r=new Random();
        int caz=r.nextInt(2);
        if(caz==0){
            //citeste un numar:
            int a=Integer.parseInt(JOptionPane.showInputDialog("a="));
            n=new Numar(a);
        }else
        {
            //citeste doua numere:
            int a=Integer.parseInt(JOptionPane.showInputDialog("a="));
            int b=Integer.parseInt(JOptionPane.showInputDialog("b="));
            n=new DouaNumere(a,b);
        }
        //Afisare:
        n.afisare();//Numai la executie se stie care versiune de metoda
                    //afisare() se va apela
    }
}

```

```
}  
}
```

7. Să se construiască clasa Cerc, ce are ca variabilă de instanță privată, un număr întreg r, ce reprezintă raza unui cerc. În această clasă avem ca metode:

- constructorul, ce face inițializarea razei;
- getRaza(), ce returnează raza;
- calculArie(), ce returnează aria cercului;
- suntEgale(), ce are ca parametru un Cerc c, și care returnează true dacă cercul curent este egal cu cercul c (au aceeași rază).
- afisare(), ce afișează raza cercului.

Din clasa Cerc se va deriva clasa CercExtins, în care se vor adăuga ca variabile de instanță x și y: coordonatele centrului și se vor redefini metodele suntEgale() (cercurile sunt egale când au aceeași rază și aceleași coordonate ale centrului), și afisare() (pe lângă rază, va afișa și coordonatele centrului)

Scrieți și o clasă de test pentru clasa CercExtins.

```
class Cerc  
{  
    private int raza;  
    public Cerc(int x)  
    {  
        raza=x;  
    }  
    public int getRaza()  
    {  
        return raza;  
    }  
    public double calculArie()  
    {  
        return Math.PI*raza*raza;  
    }  
    public boolean suntEgale(Cerc c)  
    {  
        if(this.raza==c.raza)return true;  
        else return false;  
    }  
    public void afisare()  
    {
```

```
        System.out.println("raza="+raza);
    }

}

class CercExtins extends Cerc
{
    private int x;
    private int y;
    public CercExtins(int r,int x0, int y0 )
    {
        super(r);
        x=x0;
        y=y0;
    }
    public boolean suntEgale(CercExtins c)
    {
        if(((this.getRaza()==c.getRaza())&&(this.x==c.x)&&(this.y==c.y))
        return true;
        else return false;
    }
    public void afisare()
    {
        System.out.println("raza="+this.getRaza());
        System.out.println("x="+x);
        System.out.println("y="+y);
    }
}

class TestCercExtins
{
    public static void main (String args[])
    {
        CercExtins c=new CercExtins(3,0,1);
        System.out.println("Aria= "+c.calculArie());
        CercExtins c1=new CercExtins(3,0,10);
        System.out.println("Sunt egale= "+c.suntEgale(c1));
    }
}
```

8. Folosind cele două clase anterioare, Cerc și CercExtins să se dezvolte o aplicație în care se vor citi N cercuri (de tipul Cerc sau CercExtins), ce se memorează într-un vector. Citirea unui obiect de tip Cerc sau CercExtins este dată de valoarea 0 sau 1 a unui număr aleator generat.

```
import java.util.*;
import javax.swing.*;
class AfisareCercuri
{
    public static void main (String args[])
    {
        final int N=3;//numarul de cercuri
        Cerc c[]=new Cerc[N];//vectorul de obiecte Cerc (clasa de
                                //baza)
        Random r=new Random();
        int i;
        for(i=0;i<N;i++){
            int caz=r.nextInt(2);
            if(caz==0){
                //citeste un Cerc:
                String s_raza=JOptionPane.showInputDialog("raza=");
                c[i]=new Cerc(Integer.parseInt(s_raza));
            }else{
                //citeste un CercExtins:
                String s_raza;
                s_raza=JOptionPane.showInputDialog("raza=");
                String s_x=JOptionPane.showInputDialog("x=");
                String s_y=JOptionPane.showInputDialog("y=");
                c[i]=new CercExtins(Integer.parseInt(s_raza),
                                    Integer.parseInt(s_x),
                                    Integer.parseInt(s_y));
            }
        }
        //Afisare vector:
        for(i=0;i<N;i++)
            c[i].afisare();
    }
}
```

Cap. 6

Fișiere

1. Se dă un fișier text în care este memorată o matrice de numere întregi (în fiecare linie a matricii se găsește același număr de valori întregi, separate prin spații). Să se copieze datele din fișierul text într-o matrice și să se afișeze aceasta.

```
import java.io.*;
import java.util.*;
class FisierInMatrice
{
    public static void main(String args[])
    {
        FileReader fr=null;
        BufferedReader bfr=null;
        int a[][]=new int[100][100]; //dim. acoperitoare
        int nL=0; //nr linii
        int nC=0; //nr. coloane
        int i;
        try{
            fr=new FileReader("matrice.txt");
            bfr=new BufferedReader(fr);
            String s=bfr.readLine();
            StringTokenizer tk=new StringTokenizer(s);
            nC=tk.countTokens(); //nr. de coloane
            //copiez prima linie de numere in matricea a[][]:
            for(i=0;i<nC;i++)
                a[0][i]=Integer.parseInt(tk.nextToken());
            nL++;
            for(;;){
                s=bfr.readLine();
                if(s==null)break; //sfarsit de fisier
                //copiez linia curenta de numere in matrice:
                tk=new StringTokenizer(s);
                for(i=0;i<nC;i++)
                    a[nL][i]=Integer.parseInt(tk.nextToken());
                nL++;
            }
            bfr.close();
        }
    }
}
```

```
        fr.close();
    }catch(IOException e){
        System.out.println("Eroare fisier");
        System.exit(1);
    }
    //Afisarea matricii:
    int j;
    for(i=0;i<nL;i++){
        for(j=0;j<nC;j++){
            System.out.print(a[i][j]+" ");
            System.out.println();
        }
    }
}
```

2. Să se copieze o matrice de numere întregi într-un fișier text.

```
import java.io.*;
class MatriceInFisier
{
    public static void main(String args[])
    {
        int a[][]={{1,2,3},
                    {-1,-2,-3}};
        FileWriter fw=null;
        BufferedWriter bfw=null;
        int i,j;
        try{
            fw=new FileWriter("copieMatrice.txt");
            bfw=new BufferedWriter(fw);
            for(i=0;i<a.length;i++){
                //scriu linia i a matricii in fisier:
                String s="";
                for(j=0;j<a[0].length;j++){
                    s=s+a[i][j]+" ";
                }
                bfw.write(s,0,s.length()-1);
                //trec la linia urmatoare:
                bfw.newLine();
            }
            bfw.close();
            fw.close();
        }
```

```

    }catch(IOException e){
        System.out.println("Eroare fisier");
        System.exit(1);
    }

}

} //main
}

```

3. Se citește un număr natural N . Să se genereze un fișier text ce conține N linii, în fiecare linie se găsesc N numere naturale aleatoare cuprinse între 0 și 255; numerele sunt separate între ele în cadrul unei linii din fișier, prin câte un spațiu. (În acest mod generăm în fișierul text, o imagine ce cuprinde $N*N$ pixeli. Fiecare pixel este memorat pe 8 biți, deci este o imagine în tonuri de gri).

```

import javax.swing.*;
import java.io.*;
import java.util.*;
class GenerareFisier
{
    final static int GAMA=256;
    public static void main(String args[])
    {
        int N=Integer.parseInt(JOptionPane.showInputDialog("N="));
        FileWriter fw=null;
        BufferedWriter bfw=null;
        try{
            fw=new FileWriter("imagine.txt");
            bfw=new BufferedWriter(fw);
            //folosim numere aleatoare:
            Random r=new Random();
            //scrie N linii in fisier:
            for(int i=0;i<N;i++){
                //Scrie linia i:(aceasta linie contine N numere aleatoare)
                //Construim linia i:
                String linie="";
                for(int j=0;j<N;j++){
                    linie=linie+r.nextInt(GAMA)+" ";
                }
                //O scrie efectiv in fisier:
                bfw.write(linie,0,linie.length());
                bfw.newLine();
            }
        }
    }
}

```



```
    }  
    bfw.close();  
    fw.close();  
}catch(IOException e){  
    System.out.println(e);  
    System.exit(1);}  
}  
}
```

4. Se da fișierul text test.txt, aflat în directorul curent.. Să se rescrie acest fișier, convertind literele mici în litere mari, restul caracterelor rămânând neschimbate.

```
import java.io.*;  
class FisierTextLitereMari  
{  
    public static void main(String args[])  
    {  
        /*Algoritm:  
        - copiem fișierul initial, cu litere mari intr-un fișier: temp.txt  
        - stergem fișierul initial  
        - redenumim temp.txt cu numele fișierului initial  
        (Metodele delete() si renameTo() se găsesc în clasa File.)  
        */
```

```
        File f=null;  
        File temp=null;  
        FileReader fr=null;  
        FileWriter fw=null;  
        try{  
            f=new File("test.txt");  
            fr=new FileReader(f);  
            temp=new File("temp.txt");  
            fw=new FileWriter(temp);  
            for(;;){  
                int cod=fr.read();  
                if(cod==-1)break;//s-a terminat fișier  
                char c=(char)cod;  
                char cMare=Character.toUpperCase(c);  
                fw.write(cMare);  
            }  
        }
```

```
fr.close();
fw.close();
boolean OK=f.delete();
if(OK)temp.renameTo(f);
}catch(IOException e){
    System.out.println(e);
}catch(SecurityException se){
    //pentru metodele din clasa File
    System.out.println(se);
    System.exit(1);
}
}
}
}
```

5. Să se scrie o aplicație ce implementează un test grilă. Întrebările și variantele de răspuns sunt memorate într-un fișier text ce are următoarea structură:

- Fiecare întrebare are o singură linie;
 - Sunt 2 variante de răspuns, fiecare ocupă o singură linie. O singură variantă este cea corectă și ea este totdeauna plasată după textul întrebării (este deci prima variant de răspuns din cele doua);
- Un exemplu de fișier text ce conține două întrebări:

Care este cel mai înalt munte din România ?

Moldoveanu

Negoiu

Care este capitala Spaniei ?

Madrid

Barcelona

În cadrul aplicației sunt prezentate în ordine aleatoare toate întrebările din fișierul text, pentru fiecare întrebare afișându-se cele doua variante de răspuns, de asemenea în ordine aleatoare. La afișarea pe monitor, variantele vor fi numerotate cu 1. sau cu 2. Programul citește de la utilizator numărul răspunsului pe care acesta îl consideră corect. După preluarea răspunsurilor pentru toate întrebările din fișier, se va calcula și afișa o notă ce reflectă corectitudinea răspunsurilor date de utilizator. Se va afișa de asemenea și durata completării testului, în secunde.

Pentru citirea de la tastatură se va dezvolta, separat, clasa CR.

```
import java.util.*;
import java.io.*;
class TestGrila
{
    public static void main(String args[]){
        int N = 0; // numarul de intrebari din test
        CR cr = new CR();
        String numeF = "intrebari.txt";
        FileReader fr = null;
        BufferedReader bfr = null;
        String s[]=new String[1000];// copiem intrebarile intr-un vector
                                   // cu dimensiune acoperitoare
        // incarcam toate liniile din fisier, in vectorul s[:
        try{
            fr = new FileReader(numeF);
            bfr = new BufferedReader(fr);
            // citim toate liniile
            for(;;){
                String linieCrt = bfr.readLine();
                if (linieCrt == null) break;
                s[N] = linieCrt;
                N++;
            }
            bfr.close();
            fr.close();
        }
        catch(IOException e){
            System.out.println(e);
            System.exit(1);
        }
        int nrTotalIntrebari=N/3;
        // Sunt N/3 intrebari distincte in fisier
        // construim vectorul de ordine aleatoare de prezentare
        // a intrebarilor:
        int ordine[]=new int[nrTotalIntrebari];
        int i;
        for (i = 0; i < nrTotalIntrebari; i++)
            ordine[i]=i;
        // Amestecam elementele vectorului ordine:
        amesteca(ordine);
    }
}
```

```
Random r = new Random();
int corecte = 0;
long tStart = System.currentTimeMillis();
for (i = 0; i < nrTotalIntrebari; i++){
// tiparim intrebarea si variantele de raspuns
// afisam intrebarea de pe pozitia i
System.out.println(s[3*ordine[i]]);
char raspCorect;
int modAfisRasp = r.nextInt(2);
//Daca modAfisRasp este 0: se afiseaza intai varianta 1
// de raspuns, si apoi varianta 2
//Daca modAfisRasp este 1: se afiseaza intai varianta 2
// de raspuns, si apoi varianta 1
if (modAfisRasp == 0){
    System.out.println("1. "+s[3*ordine[i]+1]);
    System.out.println("2. "+s[3*ordine[i]+2]);
    raspCorect='1';
}
else {
    System.out.println("1. "+s[3*ordine[i]+2]);
    System.out.println("2. "+s[3*ordine[i]+1]);
    raspCorect='2';
}
char raspUser;
for(;;){
    System.out.print("Raspuns = (1,2) : ");
    raspUser = cr.readChar();
    if((raspUser=='1')||(raspUser=='2'))break;
}
if (raspUser == raspCorect) corecte++;
System.out.println();
} //end for
long tStop = System.currentTimeMillis();
int t = (int)(tStop - tStart)/1000;
System.out.println();
System.out.println("Numarul total de intrebari : "
+nrTotalIntrebari);
System.out.println("Raspunsuri corecte : "+corecte);
System.out.println("Durata test : "+t+" secunde.");
double notaCalculata=10.0*corecte/nrTotalIntrebari;
```

```
//Afisam nota cu o singura zecimala:
double nota=((int)(notaCalculata*10))/10.0;
String s1 = ""+nota;
System.out.println("Nota la test : "+nota);
}
private static void amesteca(int ordine[]){
    Random r = new Random();
    int i;
    for (i = 0; i<ordine.length; i++){
        int index1 = r.nextInt(ordine.length);
        int index2 = r.nextInt(ordine.length);
        int aux = ordine[index1];
        ordine[index1]=ordine[index2];
        ordine[index2]=aux;
    }
}
}
}

//Clasa CR, folosita pt. a citi de la tastatura:
class CR
{
    private BufferedReader buff_reader;//obiect folosit pt. a citi
    // o linie de caractere
    public CR()
    {
        //creaza obiectul buff_reader ce are capacitatea de
        // a citi o linie de caractere.
        buff_reader=new BufferedReader(new InputStreamReader
                                     (System.in));
    }

    //Citeste o linie de la tastatura:
    public String readString()
    {
        String s="";
        try{
            s=buff_reader.readLine();
            //in clasa BufferedReader exista metoda readLine()
        }
    }
}
```

```

    catch(IOException e){
        System.out.println(e);
        System.exit(1);
    }
    return s;
}

```

```

//Citeste un int de la tastatura:
public int readInt()
{
    String s=this.readString();
    int nr=Integer.parseInt(s);
    return nr;
}

```

```

//Citeste un char de la tastatura:
public char readChar()
{
    String s=this.readString();
    return s.charAt(0);
}

```

```

} //end class CR

```

6. Se citește un număr natural N. Se citesc numele a N fișiere text. Să se afișeze în care fișier apare de cele mai multe ori caracterul 'a'.

```

import java.io.*;
import javax.swing.*;
class FisierMaximAparitii
{
    public static void main(String args[])
    {
        int N=Integer.parseInt(JOptionPane.showInputDialog("N="));
        String numeFisiere[]=new String[N];
        int i;
        //Citim numele celor N fișiere:
        for(i=0;i<N;i++)
            numeFisiere[i]=JOptionPane.showInputDialog("nume = ");
        //initializare:
    }
}

```

```

int nrMaxAparitii=-1;
int indexMaxAparitii=-1;//indexul fisierului in care apare de cele
                        // mai multe ori 'a'
FileReader fr=null;
try{
//parcurgem cele N fisiere:
for(i=0;i<N;i++){
fr=new FileReader(numeFisiere[i]);
int nrAparitiiFisierCrt=0;
for(;;){
int cod=fr.read();
if(cod==-1)break;//s-a terminat fisier
if((char)cod=='a')nrAparitiiFisierCrt++;
}
if(nrAparitiiFisierCrt>nrMaxAparitii){
nrMaxAparitii=nrAparitiiFisierCrt;
indexMaxAparitii=i;}
fr.close();
}
}catch(IOException e){
System.out.println(e);
System.exit(1);}
System.out.println("Nr. max. aparitii 'a' = "+nrMaxAparitii) ;
System.out.println("In fisierul: "+numeFisiere[indexMaxAparitii]);
} //main
}

```

7. Se dă un fișier text în care este memorată o imagine binară (alb- negru) ce conține $N1 \times N1$ pixeli (valori de 0 sau 1, separate printr-un spațiu în cadrul aceleiași linii). Valoarea $N1$ este cunoscută. Să se construiască un alt fișier text ce conține imaginea anterioară redusă la $N2 \times N2$ pixeli ($N2$ cunoscut și este un divizor al lui $N1$). Algoritmul de reducere este următorul: se calculează dimensiunea unui bloc (notată cu dimBloc) $\text{dimBloc} = N1/N2$, și apoi imaginea inițială de $N1 \times N1$ pixeli se împarte în blocuri de dimBloc linii și dimBloc coloane. Fiecare bloc va fi redus la un pixel, în matricea redusă (ce va avea dimensiunea $N2 \times N2$). Valoarea pixelului se calculează astfel: este 1 dacă numărul de pixeli de 1 din bloc este $\geq$ decât numărul de pixeli de 0 din bloc, și este 0 în caz contrar.

```
import java.io.*;
```

```
import java.util.*;
class ReduceFisier{
public static void main (String args[])
{
    final int N1=100;
    final int N2=10;
    int dimBloc=N1/N2;
    FileReader fr = null;
    BufferedReader bfr=null;
    int a[][] = new int[N1][N1];
    int i,j;
    try{
        //copiem fisierul mare,ce are N1 linii, intr-o matrice:
        fr = new FileReader("unu.txt");
        bfr = new BufferedReader(fr);
        for (i=0;i<N1;i++)
        {
            String s = bfr.readLine();
            StringTokenizer tk = new StringTokenizer(s);
            for(j=0;j<N1;j++)
            {
                String atomCrt = tk.nextToken();
                a[i][j]=Integer.parseInt(atomCrt);
            }
        }
        bfr.close();
        fr.close();
    }catch (IOException e){
        System.out.println(e);
        System.exit(1);}

    //Reducem matricea si o copiem in al doilea fisier:
    try {
        int b[][] = new int [N2][N2];
        //construim matricea redusa, b[][]:
        for (i=0;i<N2;i++)
            for (j=0;j<N2;j++){
                int contor1 = 0;
                for (int k = i*dimBloc;k<(i+1)*dimBloc;k++)
                    for (int l = j*dimBloc;l<(j+1)*dimBloc;l++)
                        if (a[k][l] == 1)contor1++;
            }
    }
```

```
        if (contor1 >= dimBloc * dimBloc / 2) b[i][j] = 1;
        else b[i][j] = 0;
    }
    //copiem matricea redusa, in fisier:
    FileWriter fw = new FileWriter ("doi.txt");
    BufferedWriter bfw = new BufferedWriter (fw);
    for (i=0; i<N2; i++){
        String st="";
        for (j=0; j<N2; j++){
            st = st + b[i][j] + " ";
        }
        bfw.write(st, 0, st.length() - 1);
        bfw.newLine();
    }
    bfw.close();
    fw.close();
} catch (IOException e){
    System.out.println(e);
    System.exit(1);
}
}
}
```

8. Să se afișeze care este cel mai lung cuvânt dintr-un fișier text dat. Se consideră că în fișier nu se află cuvinte despărțite în silabe, la cap de rând.

```
import java.io.*;
import java.util.*;
class CelMaiLungCuvant
{
    public static void main(String args[])
    {
        FileReader fr=null;
        BufferedReader bfr=null;
        //initializari:
        String cuvantMax="";
        int lungimeMax=0; //initializare pt. dimensiunea celui mai lung
                        //cuvant
        try{
            fr=new FileReader("referat.txt");
            bfr=new BufferedReader(fr);
```

```

for(;;){
    String linie=bfr.readLine();
    if(linie==null)break;//s-a terminat fisierul
    //Extragem cuvintele din aceasta linie:
    StringTokenizer tk=new StringTokenizer(linie);
    int n=tk.countTokens();
    for(int i=0;i<n;i++){
        String cuvant=tk.nextToken();
        int lungime=cuvant.length();
        if(lungime>lungimeMax){
            lungimeMax=lungime;
            cuvantMax=cuvant;
        }
    }
}
bfr.close();
fr.close();
}catch(IOException e){
    System.out.println(e);
    System.exit(1);}
System.out.println("Cel mai lung cuvant este: "+cuvantMax);
}
}

```

9. Se citește numele unui fișier text ce conține mai multe linii. Să se afișeze dacă toate liniile sunt diferite între ele sau nu.

```

import java.io.*;
import javax.swing.*;
class LiniiDiferite
{
    public static void main(String args[])
    {
        String numeF=JOptionPane.showInputDialog("nume fisier=");
        FileReader fr=null;
        BufferedReader bfr=null;
        //Vom copia toate liniile din fisier, intr-un vector de Stringuri:
        String s[]=new String[1000];//dimensiune acoperitoare
        //initializare pt. numarul de linii din fisier:
        int N=0;
        try{

```

```
fr=new FileReader(umeF);
bfr=new BufferedReader(fr);
for(;;){
    String linie=bfr.readLine();
    if(linie==null)break;//s-a terminat fisierul
    //copiem linia in vectorul s[]:
    s[N]=linie;
    N++;
}
bfr.close();
fr.close();
}catch(IOException e){
    System.out.println(e);
    System.exit(1);}
//Verificam daca vectorul s[] are toate elementele diferite:
for(int i=0;i<N-1;i++)
    for(int j=i+1;j<N;j++)
        if(s[i].compareTo(s[j])==0){
            System.out.println("Nu are toate liniile diferite !");
            return;}
System.out.println("Are toate liniile diferite !");
}
}
```

10. Să se afişeze dacă primul caracter dintr-un fişier text este egal cu ultimul.

```
import java.io.*;
class PrimulSiUltimulCaracter
{
    public static void main(String args[])
    {
        FileReader fr=null;
        //Presupunem ca fisierul are cel putin doua caractere!
        try{
            fr=new FileReader("date.txt");
            //citim primul caracter:
            int cod=fr.read();
            char primul=(char)cod;
            //citim urmatorul caracter si cu acesta initializam pe ultimul:
            cod=fr.read();
```

```

char ultimul=(char)cod;
//citim restul caracterelor, actualizand ultimul caracter:
for(;;){
    cod=fr.read();
    if(cod==-1)break;//s-a terminat fisier
    ultimul=(char)cod;
}
fr.close();
if(primul==ultimul)System.out.println("da");
else System.out.println("nu");
}catch(IOException e){
    System.out.println(e);
    System.exit(1);}
}
}

```

11. Se citesc N linii de la tastatura (N - cunoscut). Sa se copieze aceste linii intr-un fisier text.

```

import javax.swing.*.*;
import java.io.*.*;
class FisierScriePropozitii
{
    public static void main(String args[])
    {
        FileWriter fw=null;
        BufferedWriter bfw=null;
        int N=Integer.parseInt(
        JOptionPane.showInputDialog("N="));
        try{
            fw=new FileWriter("propozitii.txt") ;
            bfw=new BufferedWriter(fw);
            for(int i=0;i<N;i++){
                String s=JOptionPane.showInputDialog("linie=");
                bfw.write(s,0,s.length());//index start si lungimea ce se scrie
                bfw.newLine();//scrie ENTER
            }
            bfw.close();
            fw.close();
        }catch(IOException e){
            System.out.println("Eroare scriere fisier");
        }
    }
}

```

```
        System.exit(1);
    }
} //main
}
```

12. Se da un fisier text de cuvinte (fiecare cuvânt se afla pe o linie). Sa se construiasca un alt fisier text, ce contine cuvintele din primul fisier, sortate in ordine alfabetica.

```
import java.io.*;
class ScrieFisierSortat
{
    public static void main(String[] args)
    {
        FileReader fr=null;
        BufferedReader bfr=null;
        //Copiem toate cuvintele (liniile) intr-un vector de Stringuri:
        String st[]=new String[100]; //dimensiune acoperitoare
        int nL=0; //numarul de linii din fisier
        try
        {
            fr=new FileReader("cuvinte.txt");
            bfr=new BufferedReader(fr);
            for (;;)
            {
                String s=bfr.readLine();
                if (s==null) break;
                else st[nL]=s;
                nL++;
            }
            bfr.close();
            fr.close();
        } catch (IOException e) {
            System.out.println(e);
            System.exit(1);
        }
        //sortare bubblesort pentru vectorul st[]:
        for (;;)
        {
            boolean gata=true;
            for (int i=0; i<nL-1; i++)
            {
```

```

        if (st[i].compareTo(st[i+1])>0)
        {
            String aux=st[i];
            st[i]=st[i+1];
            st[i+1]=aux;
            gata=false;
        }
    }
    if (gata)break;
}
//copiere vector st[] in fisier:
FileWriter fw=null;
BufferedWriter bfw=null;
try
{
    fw=new FileWriter("cuvinteSortate.txt");
    bfw=new BufferedWriter(fw);
    for(int i=0;i<nL;i++){
        bfw.write(st[i],0,st[i].length());//index start si lungimea ce se scrie
        bfw.newLine();// scrie ENTER in fisier
    }
    bfw.close();
    fw.close();
}catch(IOException e){
    System.out.println(e);
    System.exit(1);}
}
}

```

13. Să se creeze prin program, un fișier text ce are următorul conținut:

0 0 ... 0

1 1 ... 1

.....

9 9 ... 9

În fiecare linie din fișier sunt câte 10 numere separate prin spații.

```
import java.io.*;
```

```
class ScrieFisier
```

```
{
```

```
    public static void main(String[] args)
```

```
{
    final int N=10; //numarul de linii din fisier
    FileWriter fw=null;
    BufferedWriter bfw=null;
    try
    {
        fw=new FileWriter("numere.txt");
        bfw=new BufferedWriter(fw);
        for(int i=0;i<N;i++){
            //Construim linia curenta:
            String s="";
            for(int j=0;j<N;j++){
                s=s+i+" ";
            }
            bfw.write(s,0,s.length());
            bfw.newLine();// scrie ENTER in fisier
        }
        bfw.close();
        fw.close();
    }catch(IOException e){
        System.out.println(e);
        System.exit(1);}
}
```

14. Să se calculeze câte valori nule sunt într-un fişier binar de octeţi al cărui nume se citeşte de la tastatură.

```
import java.io.*;
class MaximOctet
{
    public static void main(String args[])
    {
        FileInputStream fi=null;
        String numeF=JOptionPane.showInputDialog("nume fisier=");
        int contor=0; //initializare nr. valori nule
        try{
            fi=new FileInputStream(numeF);
            for(;;){
                int cod=fi.read();
                if(cod==-1)break; //s-a terminat fisierul
                if((byte)cod==0)contor++;
            }
        }
```

```
        fi.close();
    }catch(IOException e){
        System.out.println(e);
        System.exit(1);
    }
    System.out.println("nr. valori nule = "+contor);
}
}
```

15. Să se calculeze maximul dintr-un fișier binar de octeți.

```
import java.io.*;
class MaximOctet
{
    public static void main(String args[])
    {
        FileInputStream fi=null;
        String numeF=JOptionPane.showInputDialog("nume fisier=");
        byte max=0; //initializare maxim
        try{
            fi=new FileInputStream(numeF);
            for(;;){
                int cod=fi.read();
                if(cod==-1)break; //s-a terminat fisierul
                byte b=(byte)cod;//octetul curent citit
                if(b>max)max=b;
            }
            fi.close();
        }catch(IOException e){
            System.out.println(e);
            System.exit(1);
        }
        System.out.println("maxim="+max);
    }
}
```

16. Să se afișeze dacă toți octeții dintr-un fișier binar, sunt diferiți între ei sau nu.

```
import java.io.*;
class OctetiDiferiti
{
```

```
public static void main(String args[])
{
    FileInputStream fi=null;
    String numeF=JOptionPane.showInputDialog("nume fisier=");
    //Copiem toti octetii din fisier, intr-un vector:
    byte b[]=new byte[10000];
    int N=0;//numar de octeti din fisier (initializare)
    try{
        fi=new FileInputStream(numeF);
        for(;;){
            int cod=fi.read();
            if(cod==-1)break; //s-a terminat fisierul
            b[N]=(byte)cod;
            N++;
        }
        fi.close();
    }catch(IOException e){
        System.out.println(e);
        System.exit(1);
    }
    //Verificam daca toti octetii sunt diferiti:
    for(int i=0;i<N-1;i++)
        for(int j=i+1;j<N;j++){
            if(b[i]==b[j]){
                System.out.println("Nu sunt toti diferiti.");
                return;
            }
        }
    System.out.println("Sunt toti diferiti.");
}
}
```

17. Se citește un număr natural N. Se citesc N numere întregi, care se vor copia într-un fișier binar de numere întregi.

```
import java.io.*;
import javax.swing.*;
class CopiereNumereInFisier
{
    public static void main(String args[])
    {
        int N=Integer.parseInt( JOptionPane.showInputDialog("N="));
```

```

FileOutputStream fos=null;
DataOutputStream f=null;
try{
    fos=new FileOutputStream("numere.dat");
    f=new DataOutputStream(fos);
    //Citim cele N numere de la tastatura:
    for(int i=0;i<N;i++){
        int nr=Integer.parseInt(JOptionPane.
                                showInputDialog("nr="));
        //Scriem numarul in fisier:
        f.writeInt(nr);
    }
    f.close();
    fos.close();
}catch(IOException e){
    System.out.println(e);
    System.exit(1);
}
}
}

```

18. Să se calculeze maximul dintr-un fişier binar de numere întregi.

```

import java.io.*;
class MaximDinFisier
{
    public static void main(String args[])
    {
        File f=null;
        FileInputStream fis=null;
        DataInputStream dis=null;
        String numeF=JOptionPane.showInputDialog("nume fisier=");
        try{
            f=new File(numeF);
            long L=f.length();//lungimea in octeti
            int N=(int)L/4;//Numarul de intregi memorati in fisier
                        //(un int ocupa 4 octeti )
            System.out.println(L);
            fis=new FileInputStream(f);
            dis=new DataInputStream(fis);

```

```
//initializam maximul cu primul numar intreg din fisier:
int max=dis.readInt();
//Citim celelalte numere din fisier:
for(int i=1;i<=N-1;i++){
    int nr=dis.readInt();
    if(nr>max)max=nr;
}
System.out.println("maxim="+max);
dis.close();
fis.close();
//f.close(); Nu exista close() pentru File
}catch(IOException e){
    System.out.println(e);
    System.exit(1);
} catch(SecurityException se){
    //pentru metodele din clasa File
    System.out.println(se);
    System.exit(1);
}
}
}
}
```

19. Să se afișeze dacă toate numerele dintr-un fișier binar de numere întregi sunt egale între ele sau nu.

```
import java.io.*;
class SuntToateEgaleInFisier
{
    public static void main(String args[])
    {
        File f=null;
        FileInputStream fis=null;
        DataInputStream dis=null;
        String numeF=JOptionPane.showInputDialog("nume fisier=");
        try{
            f=new File(numeF);
            long L=f.length();//lungimea in octeti
            int N=(int)L/4;//Numarul de intregi memorati in fisier
                //(un int ocupa 4 octeti )
            fis=new FileInputStream(f);
            dis=new DataInputStream(fis);
```

```

//Citesc primul numar din fisier:
int nr1=dis.readInt();
boolean suntToateEgale=true;//semafor
//Citim restul numerelor si le comparam cu primul:
for(int i=1;i<N;i++){
    int nr=dis.readInt();
    if(nr!=nr1){
        suntToateEgale=false;
        break;}
}
if(suntToateEgale)
    System.out.println("Sunt toate numerele egale.");
else System.out.println("Nu sunt toate numerele egale.");
dis.close();
fis.close();
}catch(IOException e){
    System.out.println(e);
    System.exit(1);
}catch(SecurityException se){//pentru metodele din clasa File
    System.out.println(se);
    System.exit(1);
}
}
}
}

```

20. Să se calculeze câte numere pozitive sunt prezente într-un fişier binar de numere întregi.

```

import java.io.*;
class CateNumerePozitiveInFisier
{
    public static void main(String args[])
    {
        File f=null;
        FileInputStream fis=null;
        DataInputStream dis=null;
        String numeF=JOptionPane.showInputDialog("nume fisier=");
        try{
            f=new File(numeF);
            long L=f.length();//lungimea in octeti
            int N=(int)L/4;//Numarul de intregi memorati in fisier

```

```
        //(un int ocupa 4 octeti )
        fis=new FileInputStream(f);
        dis=new DataInputStream(fis);
        int contor=0;
        //Citim numerele din fisier:
        for(int i=0;i<N;i++){
            int nr=dis.readInt();
            if(nr>=0)contor++;
        }
        System.out.println("numarul de numere pozitive="+contor);
        dis.close();
        fis.close();
    }catch(IOException e){
        System.out.println(e);
        System.exit(1);
    }catch(SecurityException se){//pentru metodele din clasa File
        System.out.println(se);
        System.exit(1);
    }
}
}
```

21. Se dă un fişier binar de numere întregi. Să se copieze toate numerele din fişier, într-un vector de numere întregi.

```
import java.io.*;
class CopiereNumereDinFisierInVector
{
    public static void main(String args[])
    {
        File f=null;
        String numeF=JOptionPane.showInputDialog("nume fisier=");
        int N=0;//numarul de numere intregi din fisier
        //(initializarea lui N este obligatorie! Altfel, eroare la
        // compilare)
        try{
            f=new File(numeF);
            long L=f.length();//lungimea in octeti
            N=(int)L/4;//Numarul de intregi memorati in fisier
        }catch(SecurityException e){
            System.out.println(e);
        }
    }
}
```

```

        System.exit(1);
    }
    //instantiere vector:
    int a[]=new int[N];
    //Citim numerele din fisier in vector:
    FileInputStream fis=null;
    DataInputStream dis=null;
    try{
        fis=new FileInputStream(f);
        dis=new DataInputStream(fis);
        for(int i=0;i<N;i++)
            a[i]=dis.readInt();
        dis.close();
        fis.close();
    }catch(IOException e){
        System.out.println(e);
        System.exit(1);
    }
    //Afisare vector:
    for(int i=0;i<N;i++)
        System.out.println(a[i]);
    }
}

```

22. Se dă un fișier binar de octeți, date.bin. Să se copieze acest fișier în NFIS fișiere binare (NFIS – contantă ce se cunoaște): date1.bin, date2.bin, date3.bin, etc., aceste fișiere conțin părți egale din fișierul inițial date.bin (cu eventuala excepție a ultimului fișier din cele NFIS). Exemplu: dacă NFIS=2, atunci prima jumătate din date.bin se copiază în date1.bin, cealaltă jumătate se copiază în date2.bin.

```

import java.io.*;
class SplitBinaryFile
{
    public static void main(String args[])
    {
        final int NFIS=3;//numar de fisiere
        final int DIM=10000;//dim. acoperitoare
        final String numeFisier="date.bin";
        FileInputStream fi=null;
    }
}

```

```
byte b[]=new byte[DIM];
int N=0;//numar de octeti din fisier (initializare)
//copiere fisier in vector de octeti:
try{
    fi=new FileInputStream(umeFisier);
    for(;;){
        int cod=fi.read();
        if(cod==-1)break; //s-a terminat fisierul
        b[N]=(byte)cod;
        N++;
    }
    fi.close();
}catch(IOException e){
    System.out.println(e);
    System.exit(1);
}

//Generare vector de nume de fisiere de iesire:
//(in exemplul folosit: date1.txt, date2.txt, date3.txt)
String nume[]=new String[N];
//Pentru a extrage din umeFisier, numele fara extensie:
StringTokenizer tk=new StringTokenizer(
    umeFisier,".");
String numeBaza=tk.nextToken();
String numeExtensie=tk.nextToken();
int i,j;
for(i=0;i<NFIS;i++){
    nume[i]=numeBaza+(i+1)+ "." +numeExtensie;
    for(i=0;i<NFIS;i++){
        System.out.println(nume[i]);
    }
}
//Construirea celor NFIS fisiere:
FileOutputStream fo=null;
//nr. de octeti ce se copiaza intr-un fisier din cele NFIS:
int dim=N/NFIS;
try{
    for(i=0;i<NFIS;i++){
        fo=new FileOutputStream(nume[i]);
        //se va copia din vectorul b[]:
        int indexStart=i*dim;
        int indexStop=indexStart+dim;
        if(i==NFIS-1)indexStop=N; //daca este ultimul fisier
```

```
    for(j=indexStart;j<indexStop;j++)
        fo.write(b[j]);
    fo.close();
}
}catch(IOException e){
    System.out.println(e);
    System.exit(1);
}
}
}
```


Cap. 7

Aplicații grafice

1. Să se scrie o aplicație ce conține patru componente grafice: `TextField` pentru afișare și trei butoane `Button`, notate cu "unu", "doi" și "trei". Cele trei butoane vor fi așezate în linie. La apăsarea unui buton se va afișa în componenta `TextField` numele butonului apăsător.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
class ScrieNumeButon
{
    public static void main(String args[])
    {
        Fereastra f=new Fereastra();
        f.setTitle("Test de butoane");
        f.setVisible(true);
    }
}
class Fereastra extends JFrame
{
    private JTextField jtf;
    private JButton jb1, jb2, jb3;
    //constructor
    public Fereastra()
    {
        setSize(300,300);
        addWindowListener(new FereastraListener());
        //construieste componente:
        jtf=new JTextField();
        //doar pt. afisare:
        jtf.setEditable(false);
        //Construim cele trei butoane:
        jb1=new JButton("unu");
        jb2=new JButton("doi");
        jb3=new JButton("trei");
        //Le plasez intr-un container intermediar JPanel:
        JPanel jp=new JPanel();
```

```

    jp.add(jb1);jp.add(jb2);jp.add(jb3);
    //Atasam fiecarui buton, obiectul de ascultare, pentru a putea fi
    //tratate evenimentele de apasare a acestor butoane:
    ClasaButoaneListener listener=new ClasaButoaneListener();
    jb1.addActionListener(listener);
    jb2.addActionListener(listener);
    jb3.addActionListener(listener);
    //adaugam componentele in containerul final, ce se extrage din
    //fereasta:
    Container containerFinal=this.getContentPane();
    containerFinal.add(jtf,"Center"); //daca lipseste, nu deseneaza nimic
    //in centru
    containerFinal.add(jp,"South");//le aseaza in partea de jos a
    //ferestrei
}
//clasa interioara:
private class FereastraListener extends WindowAdapter
{
    public void windowClosing(WindowEvent ev)
    {
        System.exit(0);
    }
}private class ClasaButoaneListener implements ActionListener
{
    //Interfata ActionListener are o singura metoda:
    public void actionPerformed(ActionEvent ev)
    {
        Object sursa=ev.getSource();
        if(sursa==jb1)jtf.setText("unu");
        else if(sursa==jb2)jtf.setText("doi");
        else if(sursa==jb3)jtf.setText("trei");
    }
}
}
}

```

2. Să se scrie o aplicație în care se afișează o fereastră ce conține patru componente grafice: JTextField pentru introducerea unui număr întreg, JTextField pentru afișare și două componente JButton. Atunci când este apăsat primul buton, se vor afișa toți divizorii numărului introdus. Când se apasă cel de-al doilea, se iese din program.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
class GraficAfisareDivizori
{
    public static void main(String args[])
    {
        Fereastra f=new Fereastra();
        f.setTitle("Calcul divizori");
        f.setVisible(true);
    }
}
class Fereastra extends JFrame
{
    private JTextField jtfDate, jtfAfisare;
    private JButton jbCalcul, jbExit;
    //constructor
    public Fereastra()
    {
        setSize(600,400);
        addWindowListener(new FereastraListener());
        //construieste componente:
        JLabel jl1=new JLabel("Numar=");
        jtfDate=new JTextField(5);//pt. introducere numar
        JPanel jp1=new JPanel();
        jp1.add(jl1); jp1.add(jtfDate);

        JLabel jl2=new JLabel("Divizorii: ");
        jtfAfisare=new JTextField(40);
        jtfAfisare.setEditable(false) ;
        JPanel jp2=new JPanel();
        jp2.add(jl2); jp2.add(jtfAfisare);

        jbCalcul=new JButton("Calcul");
        jbExit=new JButton("Exit");
        JPanel jp3=new JPanel();
        jp3.add(jbCalcul); jp3.add(jbExit);

        //Container intermediar:
        JPanel jp=new JPanel();
```

```

jp.setLayout(new GridLayout(3,1));
jp.add(jp1); jp.add(jp2); jp.add(jp3);

ClasaButoaneListener bL=new ClasaButoaneListener();
jbCalcul.addActionListener(bL);
jbExit.addActionListener(bL);
//Containerul final:
Container c=this.getContentPane();
c.add(jp,"South");
}
//inner class:
private class FereastraListener extends WindowAdapter
{
    public void windowClosing(WindowEvent ev)
    {
        System.exit(0);
    }
}
//end inner class
//inner class:
private class ClasaButoaneListener implements ActionListener
{
    public void actionPerformed(ActionEvent ev)
    {
        Object sursa=ev.getSource();
        if(sursa==jbExit)System.exit(0);
        if(sursa==jbCalcul){
            int nr=Integer.parseInt(jtfDate.getText());
            String rezultat="1, ";//primul divizor
            for(int i=2;i<=nr/2;i++)
                if(nr%i==0)rezultat=rezultat+i+", ";
            //Ultimul divizor este chiar nr:
            rezultat=rezultat+nr+ ".";
            jtfAfisare.setText(rezultat);
        }
    }
}
//end inner class
}
//end class Fereastra

```

3. Să se scrie o aplicație în care se afișează o fereastră ce conține patru componente grafice: JTextField pentru introducerea elementelor

unui vector de numere întregi (elementele sunt separate prin spații), JTextField pentru afișarea maximului din vector și două componente JButton. Atunci când este apăsat primul buton, se va afișa maximul din vectorul introdus. Când se apasă cel de-al doilea, se iese din program.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.*;
class GraficAfisareMaxim
{
    public static void main(String args[])
    {
        Fereastra f=new Fereastra();
        f.setTitle("Calcul maxim");
        f.setVisible(true);
    }
}

class Fereastra extends JFrame
{
    private JTextField jtfDate, jtfAfisare;
    private JButton jbCalcul, jbExit;
    //constructor
    public Fereastra()
    {
        setSize(600,400);
        addWindowListener(new FereastraListener());
        //construieste componente:
        JLabel jl1=new JLabel("Vector=");
        jtfDate=new JTextField(40);//pt. introducere numar
        JPanel jp1=new JPanel();
        jp1.add(jl1); jp1.add(jtfDate);

        JLabel jl2=new JLabel("Maxim: ");
        jtfAfisare=new JTextField(5);
        jtfAfisare.setEditable(false) ;
        JPanel jp2=new JPanel();
        jp2.add(jl2); jp2.add(jtfAfisare);
```

```

jbCalcul=new JButton("Calcul");
jbExit=new JButton("Exit");
JPanel jp3=new JPanel();
jp3.add(jbCalcul); jp3.add(jbExit);

//Container intermediar:
JPanel jp=new JPanel();
jp.setLayout(new GridLayout(3,1));
jp.add(jp1); jp.add(jp2); jp.add(jp3);

ClasaButoaneListener bL=new ClasaButoaneListener();
jbCalcul.addActionListener(bL);
jbExit.addActionListener(bL);
//Containerul final:
Container c=this.getContentPane();
c.add(jp,"South");
}
//inner class:
private class FereastraListener extends WindowAdapter
{
    public void windowClosing(WindowEvent ev)
    {
        System.exit(0);
    }
} //end inner class
//inner class:
private class ClasaButoaneListener implements ActionListener
{
    public void actionPerformed(ActionEvent ev)
    {
        Object sursa=ev.getSource();
        if(sursa==jbExit)System.exit(0);
        if(sursa==jbCalcul){
            //preluarea elementelor vectorului, din JTextField:
            String s=jtfDate.getText();
            StringTokenizer tk=new StringTokenizer(s);
            int N=tk.countTokens();//nr. de elemente din vector
            //initializam maximul cu primul numar :
            int max=Integer.parseInt(tk.nextToken());
        }
    }
}

```

```
        //Comparam cu restul numerelor:
        for(int i=1;i<N;i++){
            int nrCrt=Integer.parseInt(tk.nextToken());
            if(nrCrt>max)max=nrCrt;
        }
        //Afisare maxim:
        jtfAfisare.setText(""+max);
    }
}
} //end inner class
} //end class Fereastra
```

4. Să se scrie o aplicație în care se afișează o fereastră ce conține trei componente grafice: JTextArea pentru introducerea pe mai multe linii a unor numere întregi (în fiecare linie sunt mai multe numere separate prin spații), JTextField pentru afișare și JButton pentru comanda efectuării calculelor. Atunci când se apasă butonul, în JTextField se va afișa suma tuturor numerelor din JTextArea.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.*;
class AfisareSuma
{
    public static void main(String args[])
    {
        Fereastra f=new Fereastra();
        f.setTitle("Suma numerelor");
        f.setVisible(true);
    }
}
```

```
class Fereastra extends JFrame
{
    private JTextArea jta;
    private JTextField jtfAfisare;
    private JButton jbCalcul;
    //constructor
    public Fereastra()
    {
```

```

setSize(600,400);
addWindowListener(new FereastraListener());
jta=new JTextArea();
jtfAfisare=new JTextField(5);
jtfAfisare.setEditable(false) ;
JPanel jp1=new JPanel();
jp1.add(jtfAfisare);
jbCalcul=new JButton("Calcul");
JPanel jp2=new JPanel();
jp2.add(jbCalcul);

//Container intermediar:
JPanel jp=new JPanel();
jp.setLayout(new GridLayout(2,1));
jp.add(jp1); jp.add(jp2);

ClasaButoaneListener bL=new ClasaButoaneListener();
jbCalcul.addActionListener(bL);
//Containerul final:
Container c=this.getContentPane();
c.add(jta,"Center");
c.add(jp,"South");
}

//inner class:
private class FereastraListener extends WindowAdapter
{
    public void windowClosing(WindowEvent ev)
    {
        System.exit(0);
    }
}
//end inner class
//inner class:
private class ClasaButoaneListener implements ActionListener
{
    public void actionPerformed(ActionEvent ev)
    {
        String s=jta.getText();
        //Extrag numerele :

```

```
StringTokenizer tk=new StringTokenizer(s);
int N=tk.countTokens();
int suma=0;
for(int i=0;i<N;i++)
    suma=suma+Integer.parseInt(tk.nextToken());
//Afisare suma:
jtfAfisare.setText(""+suma);
}
}
} //end class Fereastra
```

5. Să se scrie o aplicație în care se afișează o fereastră ce conține trei componente grafice: JTextArea pentru introducerea unui text, JTextField pentru afișare și JButton pentru comanda efectuării calculelor. Atunci când se apasă butonul, în JTextField se va afișa numărul de caractere 'a' din textul introdus în JTextArea.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.*;
class Calcul
{
    public static void main(String args[])
    {
        Fereastra f=new Fereastra();
        f.setTitle("Calcul nr. caractere a");
        f.setVisible(true);
    }
}

class Fereastra extends JFrame
{
    private JTextArea jta;
    private JTextField jtfAfisare;
    private JButton jbCalcul;
    //constructor
    public Fereastra()
    {
        setSize(600,400);
        addWindowListener(new FereastraListener());
    }
}
```

```
jta=new JTextArea();
jtfAfisare=new JTextField(5);
jtfAfisare.setEditable(false) ;
JPanel jp1=new JPanel();
jp1.add(jtfAfisare);
jbCalcul=new JButton("Calcul");
JPanel jp2=new JPanel();
jp2.add(jbCalcul);
//Container intermediar:
JPanel jp=new JPanel();
jp.setLayout(new GridLayout(2,1));
jp.add(jp1); jp.add(jp2);
ClasaButoaneListener bL=new ClasaButoaneListener();
jbCalcul.addActionListener(bL);
//Containerul final:
Container c=this.getContentPane();
c.add(jta,"Center");
c.add(jp,"South");
}
//inner class:
private class FereastraListener extends WindowAdapter
{
    public void windowClosing(WindowEvent ev)
    {
        System.exit(0);
    }
}
//end inner class
//inner class:
private class ClasaButoaneListener implements ActionListener
{
    public void actionPerformed(ActionEvent ev)
    {
        String s=jta.getText();
        int contor=0;
        for(int i=0;i<s.length();i++)
            if(s.charAt(i)=='a')contor++;
        jtfAfisare.setText(""+contor);
    }
}
}
}
//end class Fereastra
```

6. Să se scrie o aplicație în care se afișează o fereastră ce conține următoarele componente grafice: JTextField pentru afișarea textului: *Afisare cu diferite marimi de fonturi !*, trei butoane radio JRadioButton, pentru selecția mărimii fontului cu care se scrie textul din JTextField (font mic, font mediu, font mare), și un buton JButton pentru ieșirea din program. Atunci când este activat primul buton radio, textul se va afișa în JTextField folosind font de mărime 12, când este activat al doilea buton radio, textul se va afișa cu font de mărime 18, când este activat al treilea, se va folosi font de mărime 24.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.*;
class GraficButoaneRadio
{
    public static void main(String args[])
    {
        Fereastră f=new Fereastră();
        f.setTitle("Butoane radio");
        f.setVisible(true);
    }
}

class Fereastră extends JFrame
{
    private JTextField jtfAfisare;
    private JButton jbExit;
    private JRadioButton radio1,radio2,radio3;

    //constructor
    public Fereastră()
    {
        setSize(600,400);
        addWindowListener(new FereastrăListener());
        //construieste componente:
        jtfAfisare=new JTextField
("Afisare cu diferite marimi de fonturi !");
        jtfAfisare.setEditable(false) ;
        radio1= new JRadioButton("font mic");
        radio1.setSelected(true);
```

```
radio2= new JRadioButton("font mediu");
radio2.setSelected(false);
radio3= new JRadioButton("font mare");
radio3.setSelected(false);
ButtonGroup radioGrup= new ButtonGroup();
radioGrup.add(radio1);
radioGrup.add(radio2);
radioGrup.add(radio3);
JPanel jpRadio= new JPanel();
//in JPanel nu este permisa adaugarea de ButtonGroup !
jpRadio.add(radio1);
jpRadio.add(radio2);
jpRadio.add(radio3);

jbExit=new JButton("Exit");
JPanel jpExit=new JPanel();
jpExit.add(jbExit);

//Container intermediar:
JPanel jp=new JPanel();
jp.setLayout(new GridLayout(2,1));
jp.add(jpRadio); jp.add(jpExit);

ClasaButoaneListener bL=new ClasaButoaneListener();
jbExit.addActionListener(bL);
radio1.addActionListener(bL);
radio2.addActionListener(bL);
radio3.addActionListener(bL);
//Containerul final:
Container c=this.getContentPane();
c.add(jtfAfisare,"Center");
c.add(jp,"South");
//Scriem textul in jtfAfisare cu font mic (marimea 12):
jtfAfisare.setFont(new Font("MonoSpaced",Font.ITALIC,12));
jtfAfisare.repaint();
}
//inner class:
private class FereastraListener extends WindowAdapter
{
    public void windowClosing(WindowEvent ev)
```

```
{
    System.exit(0);
}
} //end inner class
//inner class:
private class ClasaButoaneListener implements ActionListener
{
    public void actionPerformed(ActionEvent ev)
    {
        Object sursa=ev.getSource();
        if(sursa==jbExit)System.exit(0);
        else{
            String tipFont="MonoSpaced";
            int stilFont=Font.ITALIC;
            int marimeFont=12;
            if(radio2.isSelected())marimeFont=18;
            else if(radio3.isSelected())marimeFont=24;
            jtfAfisare.setFont(new Font(tipFont,stilFont,marimeFont));
            jtfAfisare.repaint();
        }
    }
}
} //end class Fereastra
```

7. Să se scrie o aplicație în care se afișează o fereastră ce conține trei componente grafice: JTextField pentru introducerea unui text, JTextField pentru afișare și un buton JButton pentru ieșirea din program. Atunci când se apasă tasta ENTER în primul JTextField, textul introdus acolo va fi copiat în al doilea JTextField.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.*;
class GraficCopiereText
{
    public static void main(String args[])
    {
        Fereastra f=new Fereastra();
        f.setTitle("Copiere text");
        f.setVisible(true);
    }
}
```

```
}  
}  
  
class Fereastra extends JFrame  
{  
    private JTextField jtfDate,jtfAfisare;  
    private JButton jbExit;  
  
    //constructor  
    public Fereastra()  
    {  
        setSize(600,400);  
        addWindowListener(new FereastraListener());  
        //construieste componente:  
        jtfDate=new JTextField(40);  
        jtfDate.setEditable(true);  
        jtfDate.addKeyListener(new ClasaTasteListener());  
        JPanel jp1=new JPanel();  
        jp1.add(jtfDate);  
  
        jtfAfisare=new JTextField(40);  
        jtfAfisare.setEditable(false);  
        JPanel jp2=new JPanel();  
        jp2.add(jtfAfisare);  
  
        jbExit=new JButton("Exit");  
        JPanel jp3=new JPanel();  
        jp3.add(jbExit);  
  
        //Container intermediar:  
        JPanel jp=new JPanel();  
        jp.setLayout(new GridLayout(3,1));  
        jp.add(jp1); jp.add(jp2); jp.add(jp3);  
  
        ClasaButoaneListener bL=new ClasaButoaneListener();  
        jbExit.addActionListener(bL);  
        //Containerul final:  
        Container c=this.getContentPane();  
        c.add(jp,"South");  
    }  
}
```

```
//inner class:
private class FereastraListener extends WindowAdapter
{
    public void windowClosing(WindowEvent ev)
    {
        System.exit(0);
    }
} //end inner class
//inner class:
private class ClasaButoaneListener implements ActionListener
{
    public void actionPerformed(ActionEvent ev)
    {
        System.exit(0);
    }
}

private class ClasaTasteListener extends KeyAdapter
{
    public void keyPressed(KeyEvent evt)
    {
        // S-a tastat ENTER ?
        if (evt.getKeyChar() == '\n') {
            jtfAfisare.setText(jtfDate.getText());
        }
    }
}
} //end class Fereastra
```

Bibliografie

1. Georgescu H., Introducere in universul Java, Ed. Tehnica, 2002
2. Keogh J., Java fara mistere, Ed. Rosetti Educational, 2006
3. Tanasa S., Olaru C., Andrei S. – Java de la 0 la expert, Ed. Polirom, 2003
4. Ene Al.- Programare orientată pe obiecte, note de curs, Universitatea din Pitești, 2007
